



Android



What is Android

- ❑ Android is a software package and Linux based operating system for mobile devices such as tablet computers and smartphones.
- ❑ It is developed by Google and later the OHA (Open Handset Alliance). Java language is mainly used to write the android code even though other languages can be used.
- ❑ The goal of android project is to create a successful real-world product that improves the mobile experience for end users.



Open Handset Alliance (OHA)

- It's a consortium of 84 companies such as google, samsung, AKM, synaptics, KDDI, Garmin, Teleca, Ebay, Intel etc.
- It was established on 5th November, 2007, led by Google. It is committed to advance open standards, provide services and deploy handsets using the Android Platform.



Features of Android

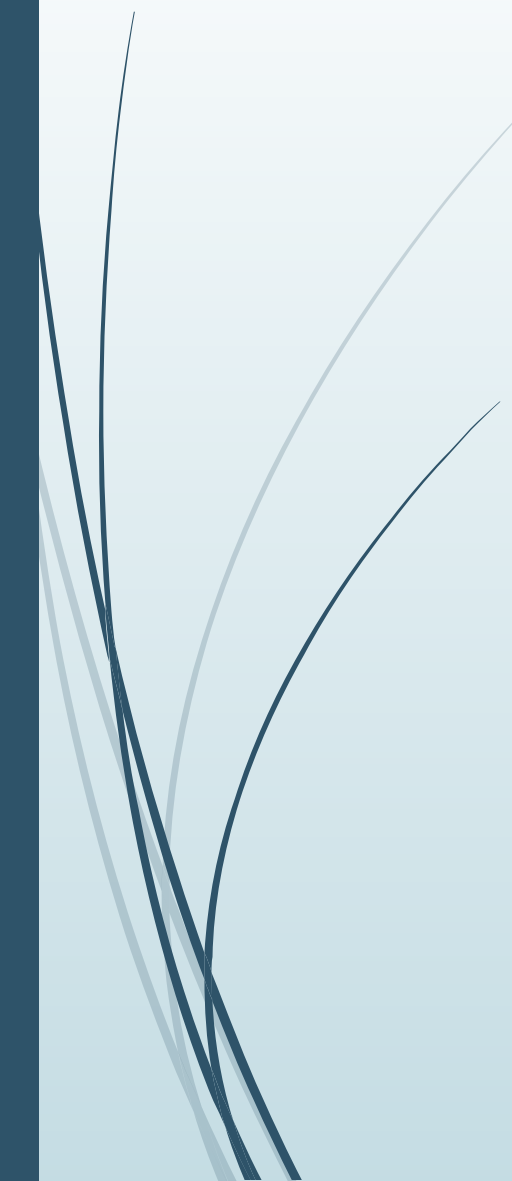
□ The important features of android are given below:

- 1) It is **open-source**.
- 2) Anyone can **customize** the Android Platform.
- 3) There are a lot of mobile applications that can be **chosen** by the consumer.
- 4) It provides many interesting features like **weather details, opening screen, live RSS** (Really Simple Syndication) feeds etc.
- 5) It provides support for **messaging** services(SMS and MMS), **web browser, storage** (SQLite), **connectivity** (GSM, CDMA, Blue Tooth, Wi-Fi etc.), **media, handset layout** etc.



Categories of Android applications

□ There are many android applications in the market. The top categories are:

1. Entertainment
 2. Tools
 3. Communication
 4. Productivity
 5. Personalization
 6. Music and Audio
 7. Social
 8. Media and Video
 9. Travel and Local etc.
- 



History of Android

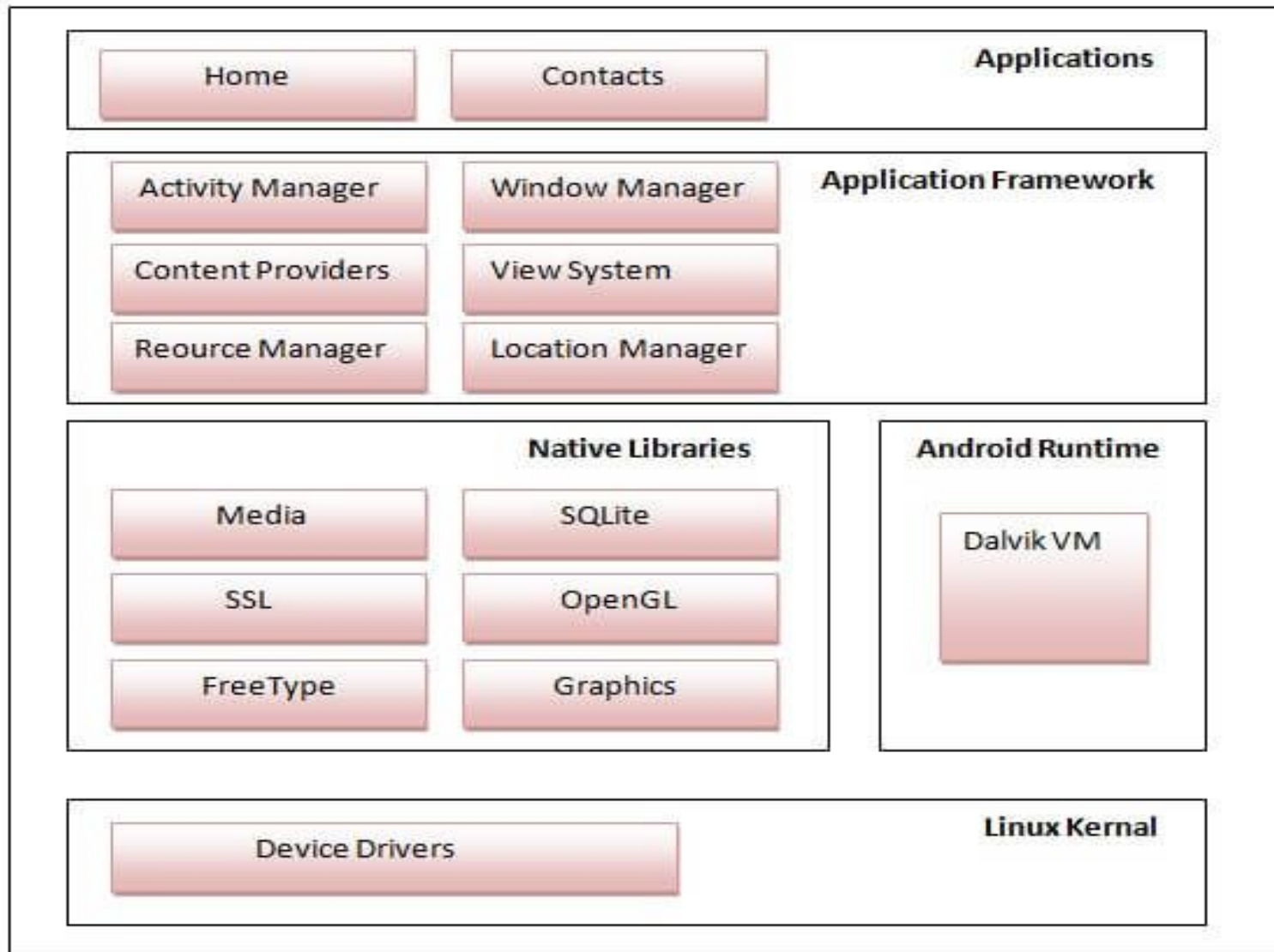


- ❑ Initially, **Andy Rubin** founded Android Incorporation in California, United States in **October, 2003**.
- ❑ In 17th August 2005, **Google** acquired android Incorporation. Since then, it is in the subsidiary of Google Incorporation.
- ❑ Originally intended for **camera** but shifted to **smart phones** later because of low market for camera only.
- ❑ Android is the nick name of Andy Rubin given by co-workers.
- ❑ In 2007, Google announces the development of android OS.
- ❑ In 2008, HTC launched the first android mobile.



Android Architecture

- ❑ **android architecture** or **Android software stack** is categorized into five parts:
- ❑ Linux kernel
- ❑ native libraries (middleware),
- ❑ Android Runtime
- ❑ Application Framework
- ❑ Applications





1) Linux kernel

- It is the heart of android architecture that exists at the root of android architecture. **Linux kernel** is responsible for device drivers, power management, memory management, device management and resource access.

2) Native Libraries

- On the top of linux kernel, there are **Native libraries** such as WebKit, OpenGL, FreeType, SQLite, Media, C runtime library (libc) etc.
- The WebKit library is responsible for browser support, SQLite is for database, FreeType for font support, Media for playing and recording audio and video formats.



3) Android Runtime

- In android runtime, there are core libraries and DVM (Dalvik Virtual Machine) which is responsible to run android application. DVM is like JVM but it is optimized for mobile devices. It consumes less memory and provides fast performance.

4) Android Framework

- On the top of Native libraries and android runtime, there is android framework. Android framework includes **Android API's** such as UI (User Interface), telephony, resources, locations, Content Providers (data) and package managers. It provides a lot of classes and interfaces for android application development.



5) Applications

- On the top of android framework, there are applications. All applications such as home, contact, settings, games, browsers are using android framework that uses android runtime and libraries. Android runtime and native libraries are using linux kernal.



Android Core Building Blocks

- ❑ An android **component** is simply a piece of code that has a well defined **life cycle** e.g. Activity, Receiver, Service etc.
- ❑ The **core building blocks** or **fundamental components** of android are activities, views, intents, services, content providers, fragments and AndroidManifest.xml.

Activity

- ❑ A **screen** through which a user can interact.
- ❑ An activity is a **class** that represents a **single screen**.
- ❑ It is like a Frame in AWT.
- ❑ Pages of application.

- 
- Contains 2 modules
 1. An **XML** file that defines user interface of the app.
 2. A **JAVA** file containing code to handle UI.
 - Starts with an intent.
 - An **Intent** is a message passing mechanism telling the android application what the user is asking it to do.

View

- A view is the **UI element** such as **button**, **label**, **text field** etc. Anything that you see is a view.

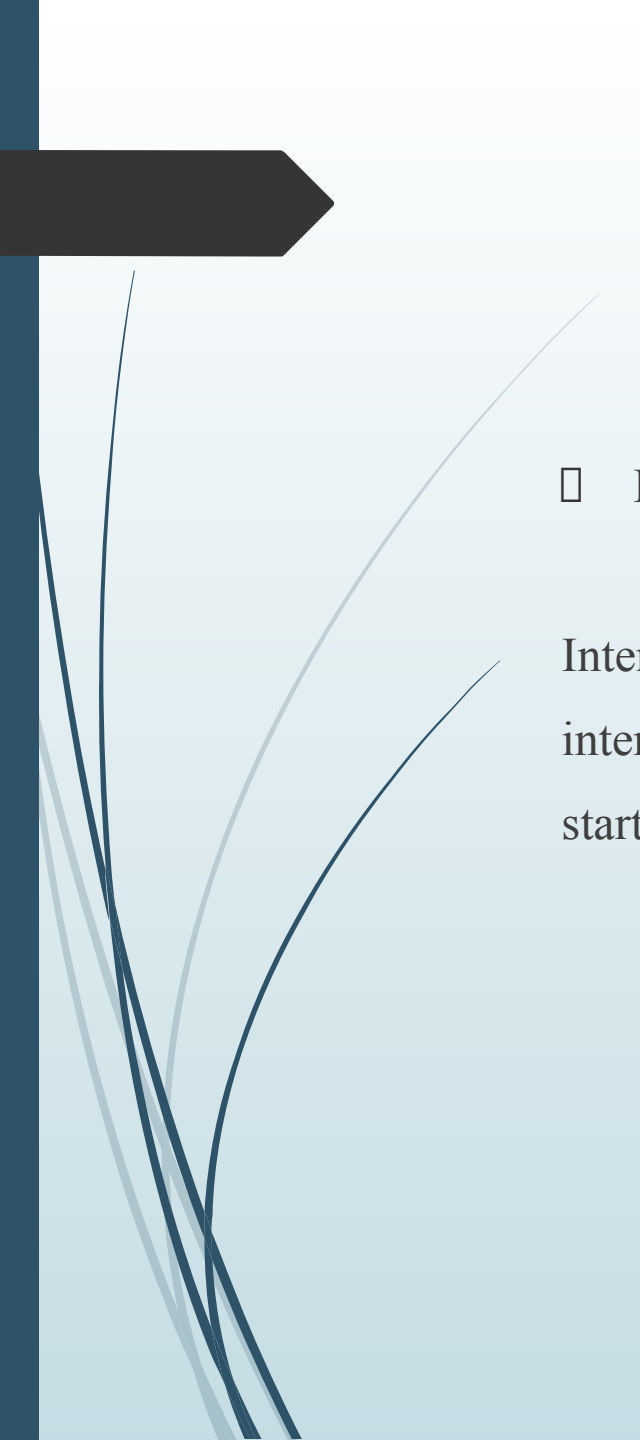
- 
- These are placed inside the layouts.
 - A group of arranged views on an activity is known as Viewgroup/Layout/Container.



Intent

□ Intent is used to **invoke components**. It is mainly used to:

1. Start the service
2. Launch an activity
3. Display a web page
4. Display a list of contacts
5. Broadcast a message
6. Dial a phone call etc.



□ For example, you may write the following code to view the webpage.

```
Intent intent=new Intent(Intent.ACTION_VIEW);  
intent.setData(Uri.parse("http://www.google.com"));  
startActivity(intent);
```




Service

- ❑ Service is a **background process** that can run for a long time.
- ❑ There are two types of services **local** and **remote**.
- ❑ Local service is accessed from **within the application** whereas remote service is accessed remotely from **other applications** running on the **same device**.

Content Provider

- ❑ Content Providers are used to share data between the applications.

Fragment

- ❑ Fragments are like **parts of activity**.
- ❑ An activity can display one or more fragments on the screen at the same time.

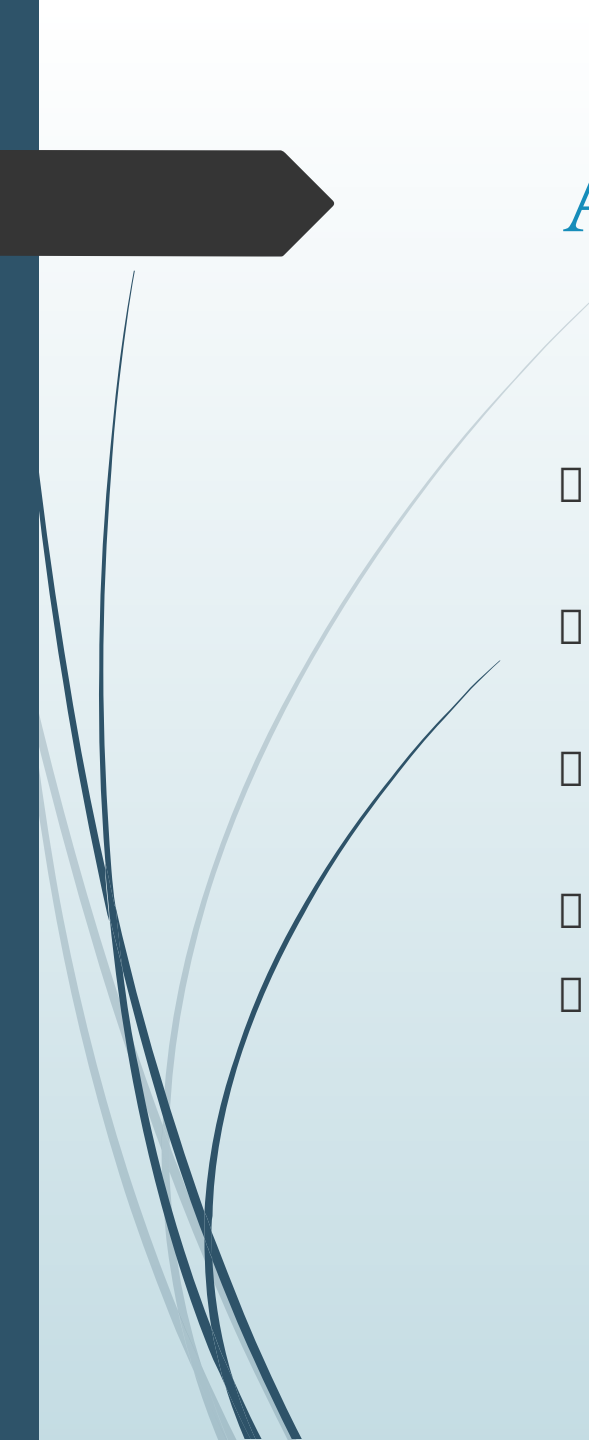


AndroidManifest.xml

- ❑ It contains **information** about activities, content providers, permissions etc.

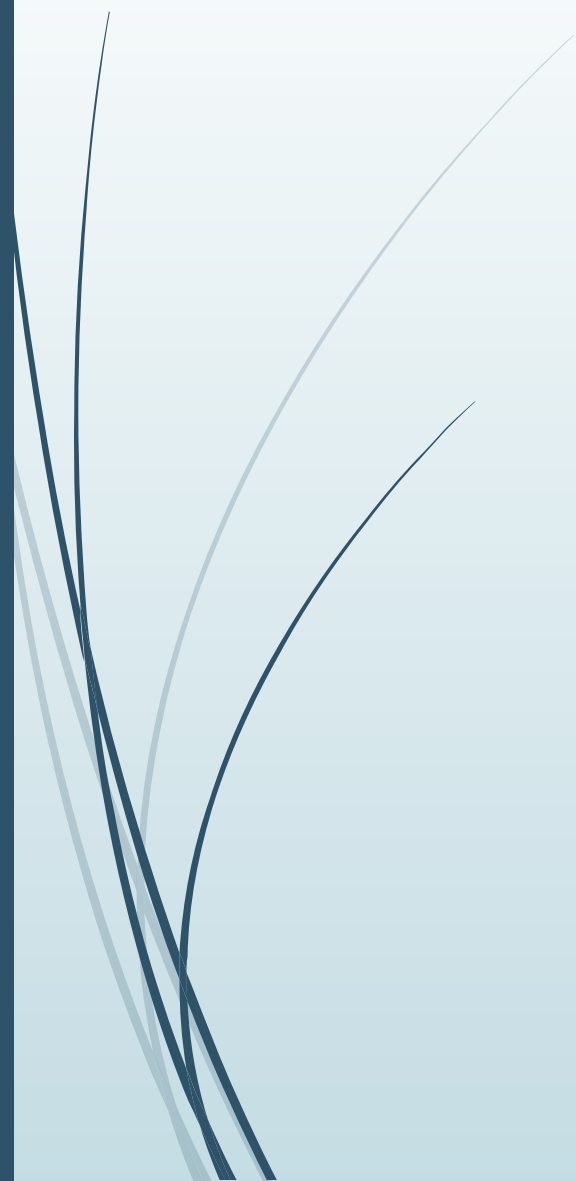
Android Virtual Device (AVD)

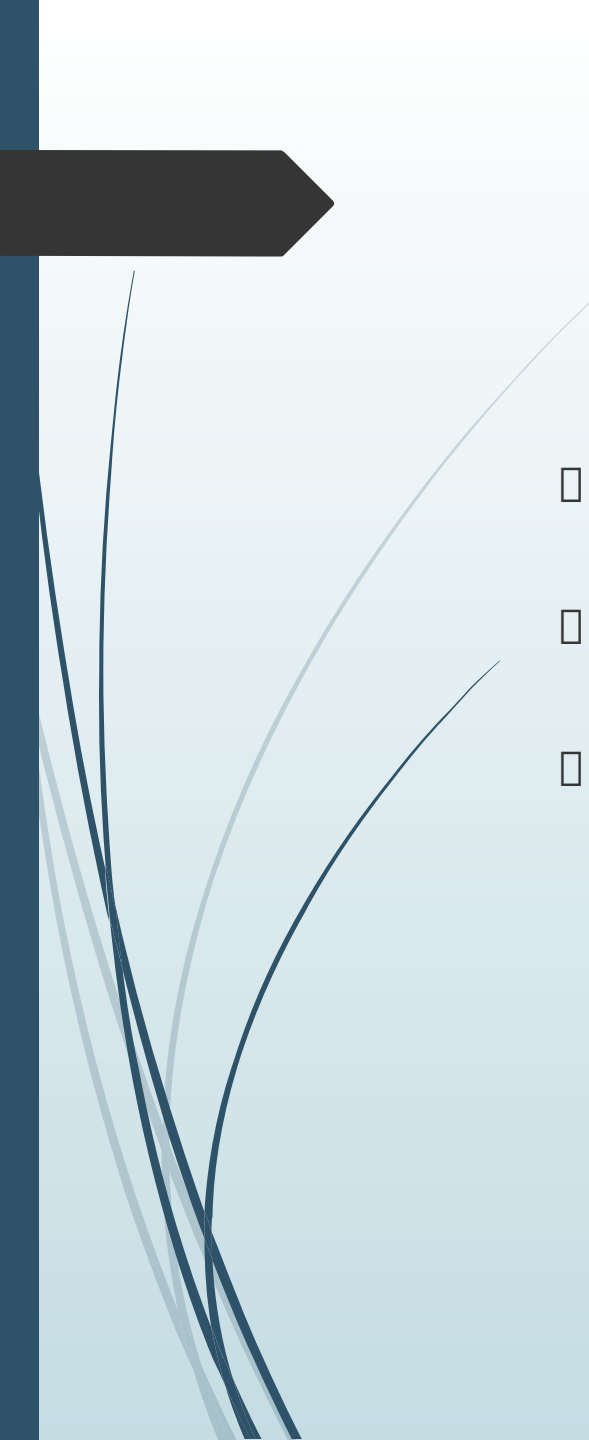
- ❑ It is used to **test** the android application without the need for mobile or tablet etc.
- ❑ It can be created in different configurations to emulate different types of real devices.



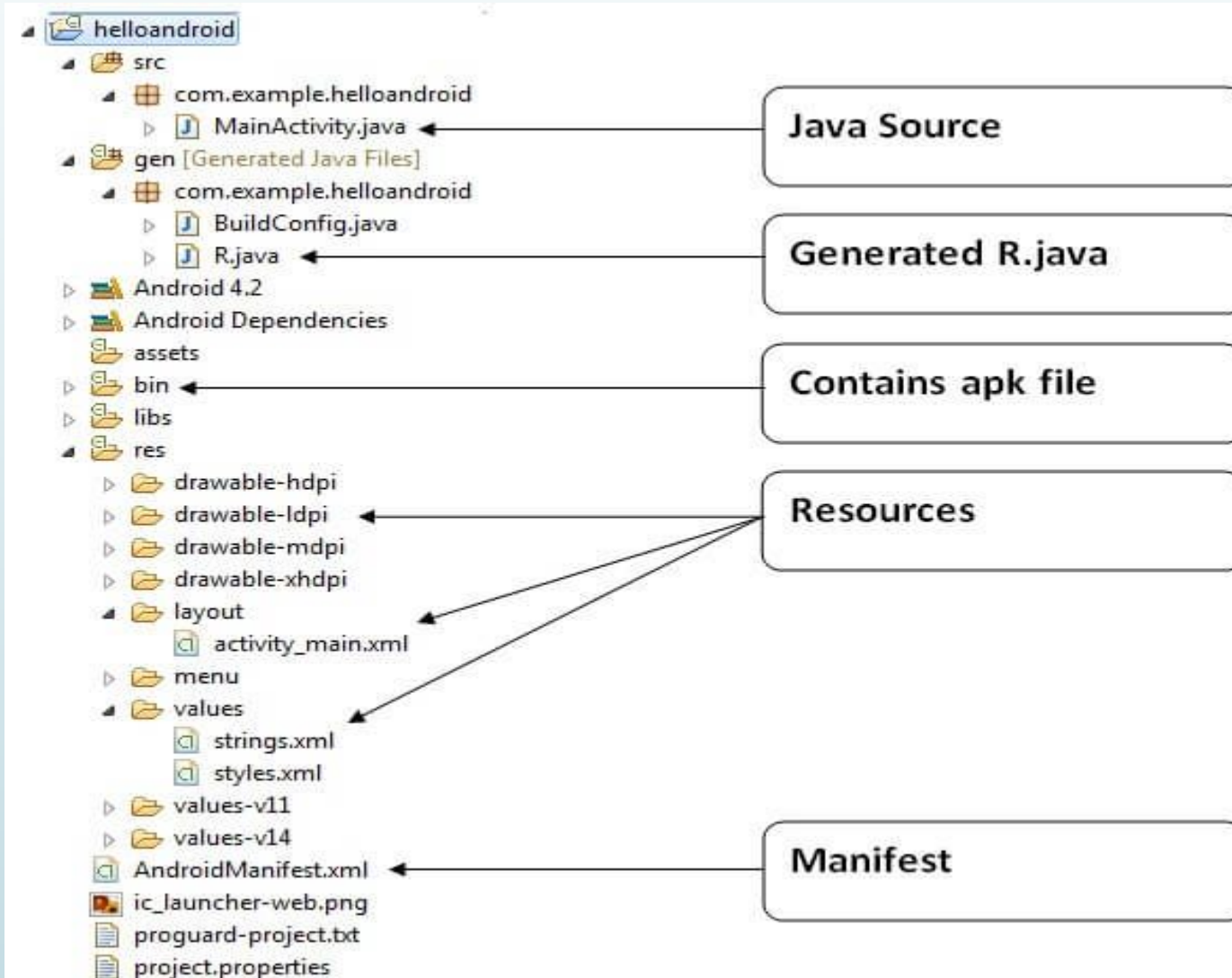
Android Emulator

- ❑ The **Android emulator** is an **Android Virtual Device (AVD)**, which represents a specific Android device.
- ❑ We can use the Android emulator as a **target device** to execute and test our Android application on our PC.
- ❑ Android emulator provides almost all the **functionality** of a real device. We can get the incoming phone calls and text messages.
- ❑ It also gives the **location** of the device and simulates different network speeds.
- ❑ Android emulator simulates rotation and other hardware sensors. It accesses the Google Play store, and much more.



- 
- ❑ Testing Android applications on emulator are sometimes faster and easier than doing on a real device.
 - ❑ For example, we can transfer data faster to the emulator than to a real device connected through USB.
 - ❑ The Android emulator comes with predefined configurations for several Android phones, Wear OS, tablet, Android TV devices.

Internal Details of Hello Android Example





Hello Android Example(.xml)

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<android.support.constraint.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
```

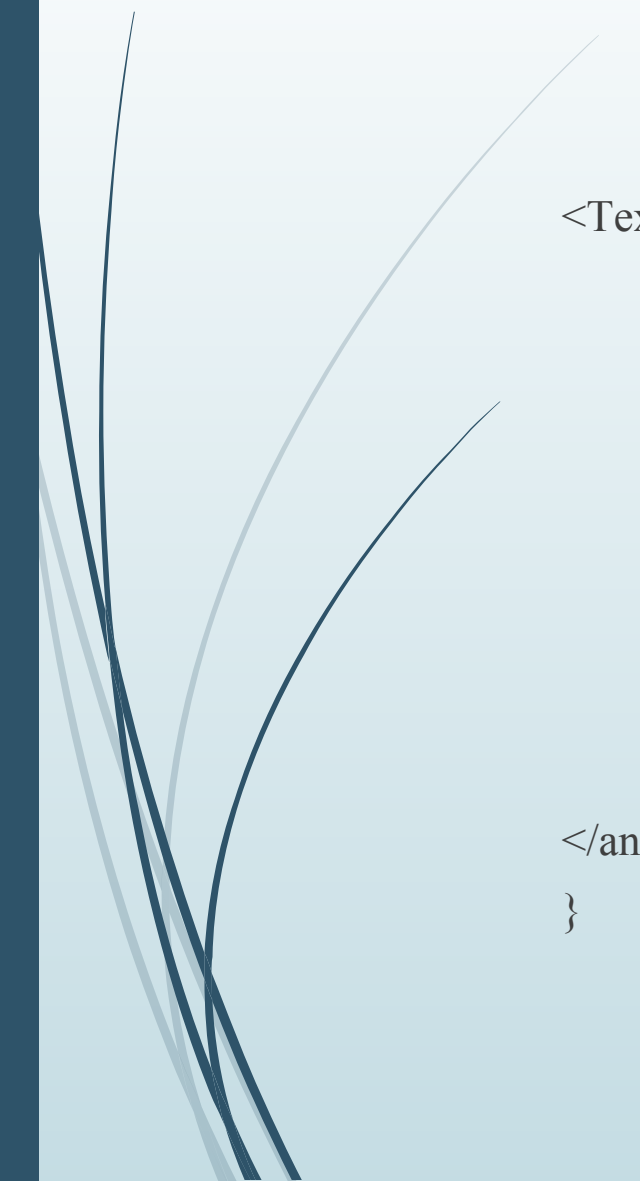
```
    xmlns:app="http://schemas.android.com/apk/res-auto"
```

```
    xmlns:tools="http://schemas.android.com/tools"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    tools:context="first.javatpoint.com.welcome.MainActivity">
```



```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello Android!"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintRight_toRightOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

</android.support.constraint.ConstraintLayout>
}
```




.java

```
package first.javatpoint.com.welcome;
```

```
import android.support.v7.app.AppCompatActivity;
```

```
import android.os.Bundle;
```

```
public class MainActivity extends AppCompatActivity { (1)
```

```
    @Override
```

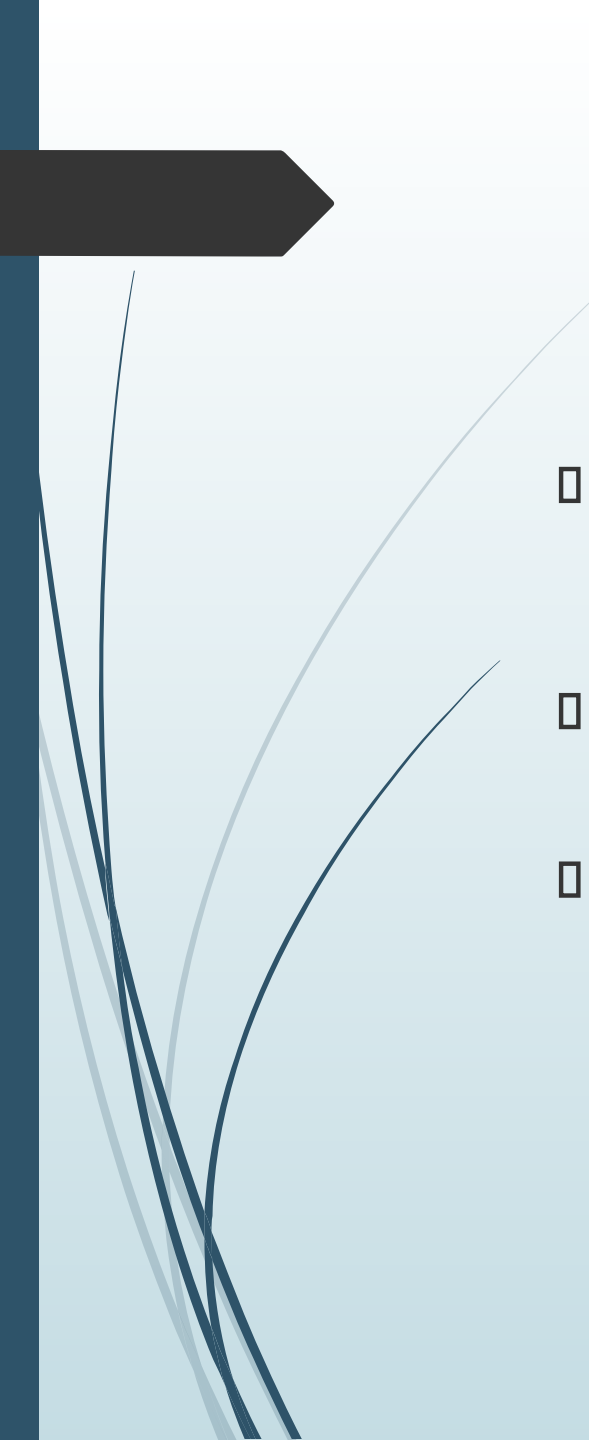
```
    protected void onCreate(Bundle savedInstanceState) { (2)
```

```
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_main); (3)
```

```
    }
```

```
}
```

- 
- ❑ **(1) Activity** is a java class that creates a default **window** on the screen where we can place different **components** such as Button, EditText, TextView, Spinner etc.
 - ❑ **(2) The onCreate method** is called when Activity class is first created.
 - ❑ **(3) The setContentView(R.layout.activity_main)** gives information about our layout resource. Here, our layout resources are defined in activity_main.xml file.



Generated R.java file

- It is the auto-generated file that contains **IDs** for all the resources of **res directory**.
- It is generated by aapt(Android Asset Packaging Tool).
- Whenever you create any component on activity_main, a corresponding ID is created in the R.java file which can be **used in the Java Source** file later.



APK File



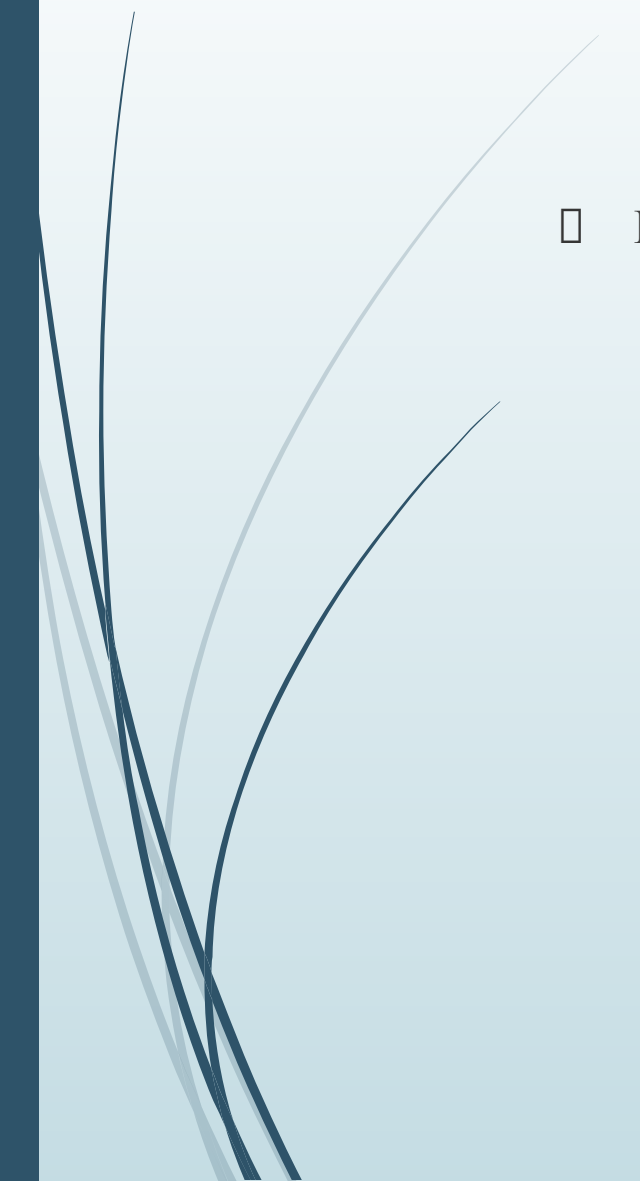
- An apk file is created by the framework automatically.
- If you want to run the android application on the mobile, transfer and install it.
- Android Studio 3.0 and higher allow you to profile and debug APKs without having to build them from an Android Studio project.

Creating an APK file

- **Build > Build Bundle(s)/APK(s) > Build APK(s)** from the toolbar menu

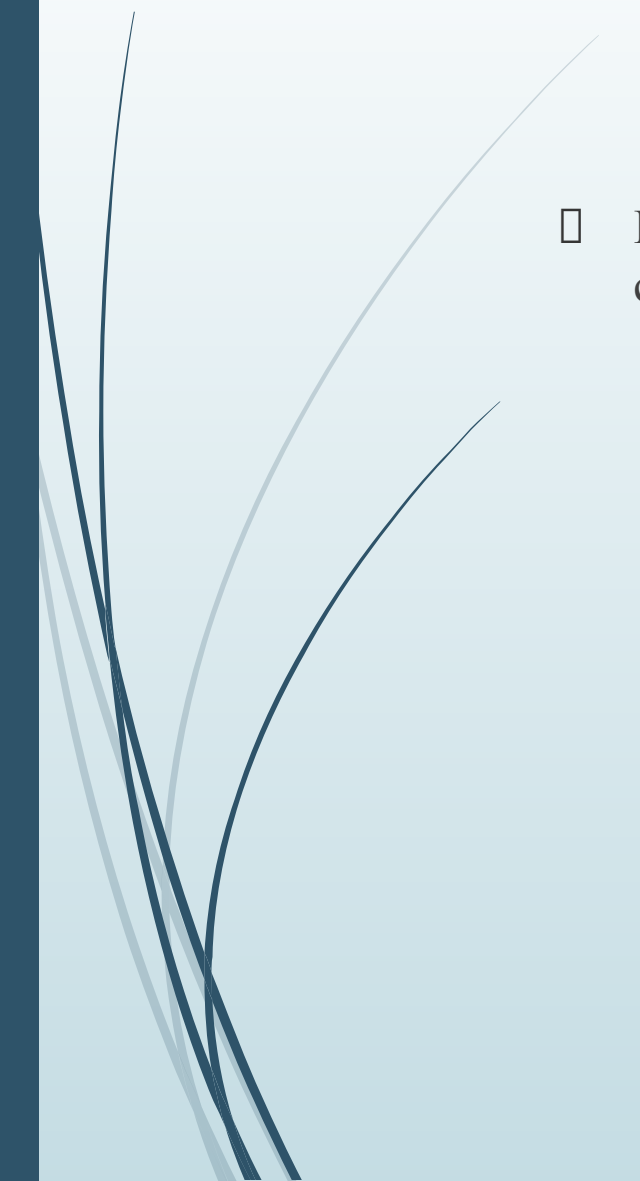


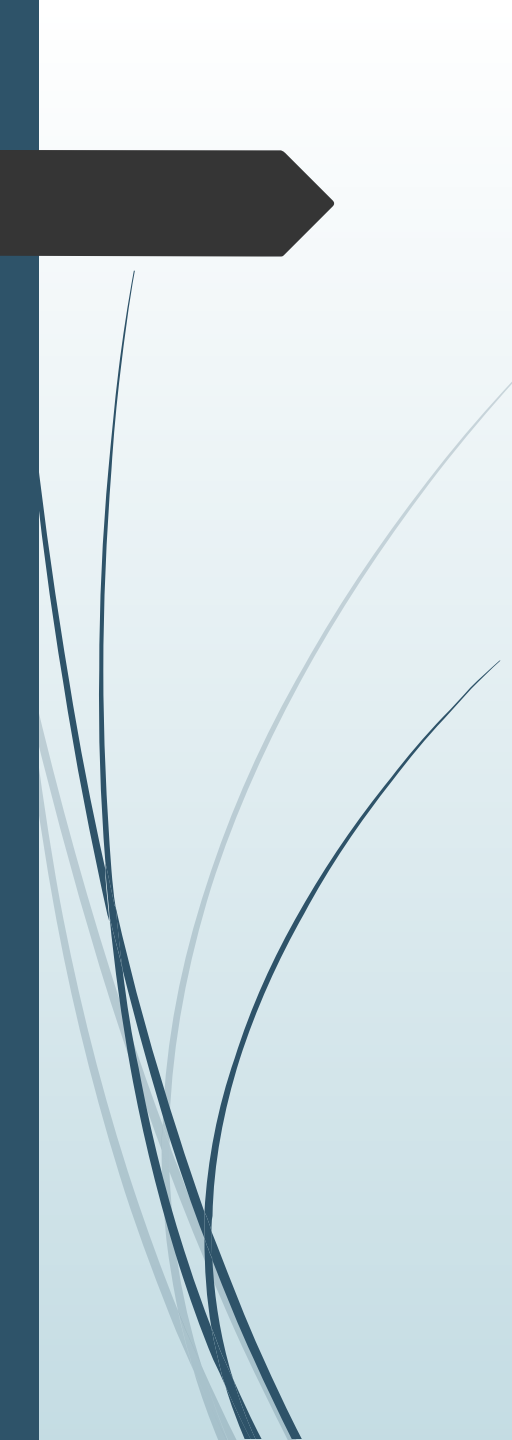
Resources

- It contains resource files including activity_main, strings, styles etc.
- 



Manifest file

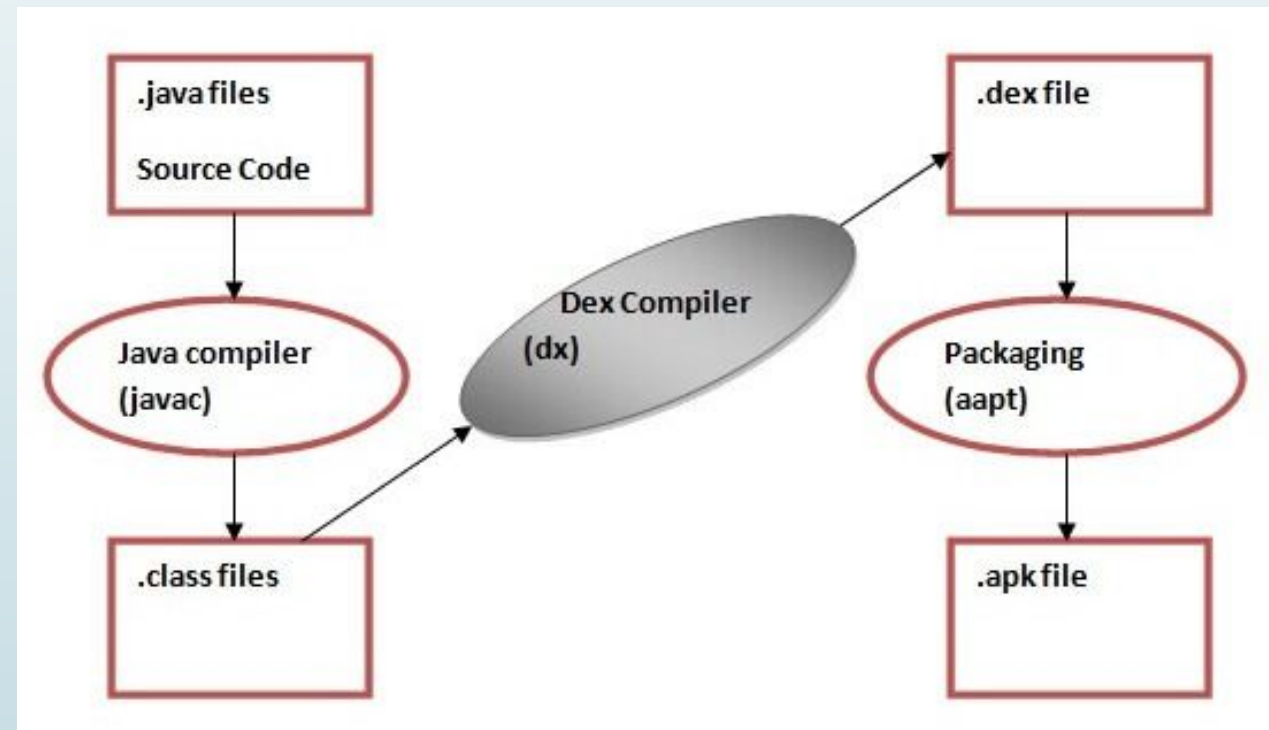
- It contains **information** about package including components such as activities, services, content providers etc.
- 

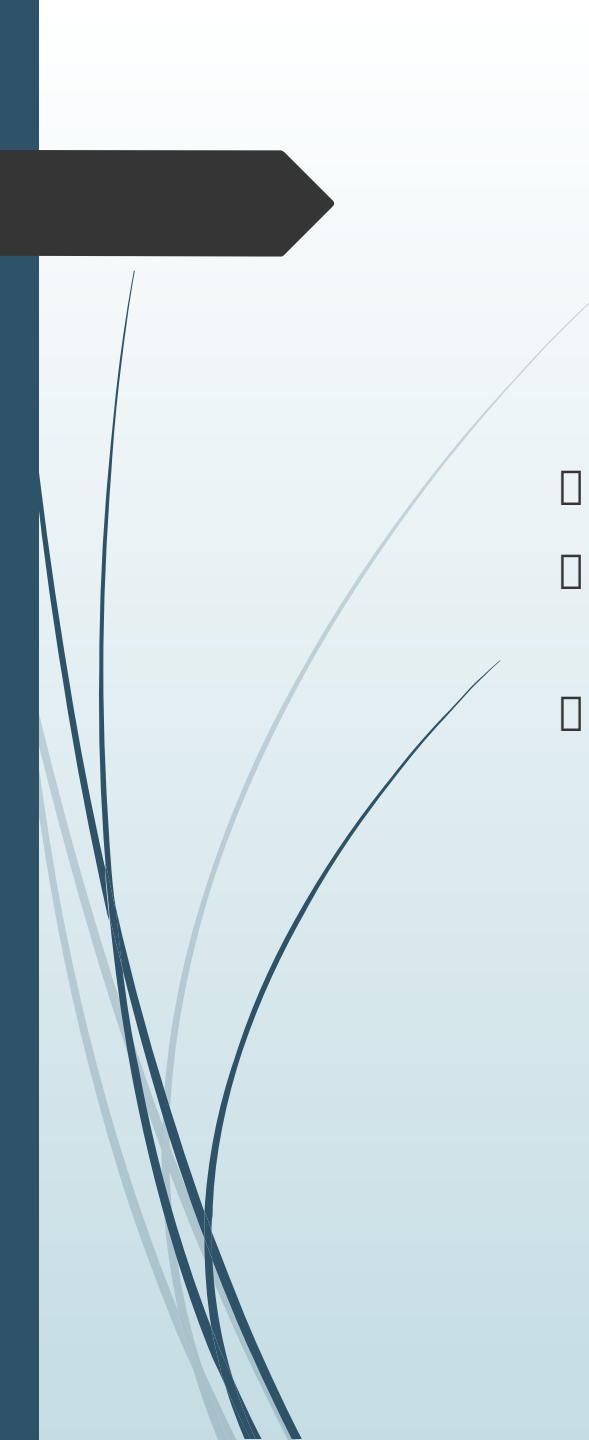


Dalvik Virtual Machine

- As we know the modern JVM is **high performance** and provides excellent **memory management**. But it needs to be optimized for low-powered handheld devices as well.
- The **Dalvik Virtual Machine (DVM)** is an android virtual machine optimized for **mobile devices**. It optimizes the virtual machine for *memory*, *battery life* and *performance*.
- The **Dex(Dalvik Executable format) compiler** converts the class files into the **.dex file** that run on the Dalvik VM. Multiple class files are converted into one dex file.

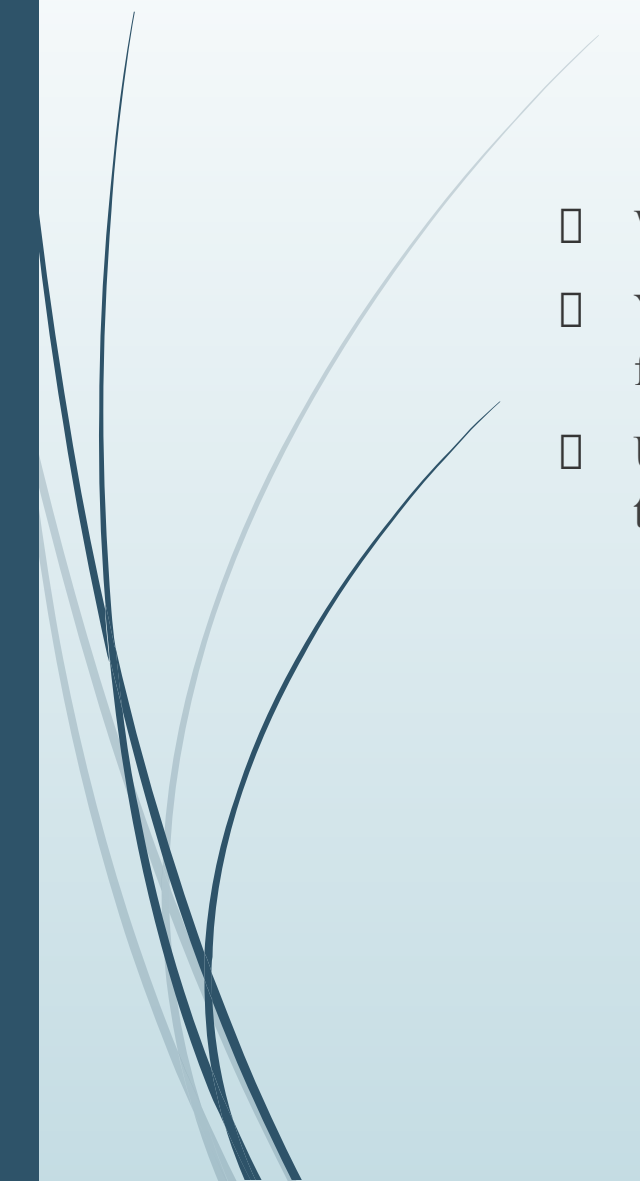
- Compiling and packaging process from the source file:



- 
- ❑ The **javac tool** compiles the java source file into the class file.
 - ❑ The **dx tool** takes all the class files of your application and generates a single .dex file. It is a platform-specific tool.
 - ❑ The **Android Assets Packaging Tool (aapt)** handles the packaging process.



App widgets

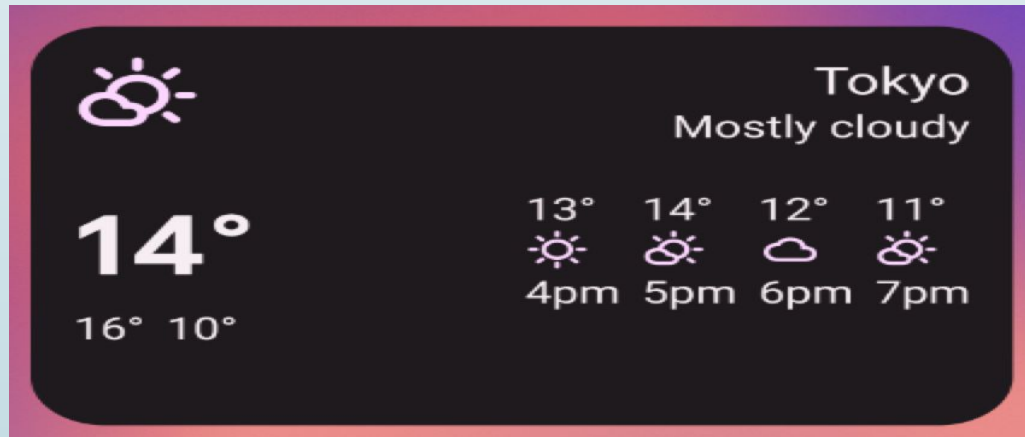


- ❑ Widgets are an essential aspect of home screen customization.
- ❑ You can imagine them as "at-a-glance" views of an app's most important data and functionality that is accessible right from the user's home screen.
- ❑ Users can move widgets across their home screen panels, and, if supported, resize them to tailor the amount of information within the widget to their preference.

Widget types

Information widgets:

- ❑ Information widgets typically display a few **crucial information elements** and **track** how that information changes over time.
- ❑ Good examples for information widgets are weather widgets, clock widgets or sports score tracking widgets.
- ❑ Touching information widgets typically launches the associated app and opens a detailed view of the widget information.





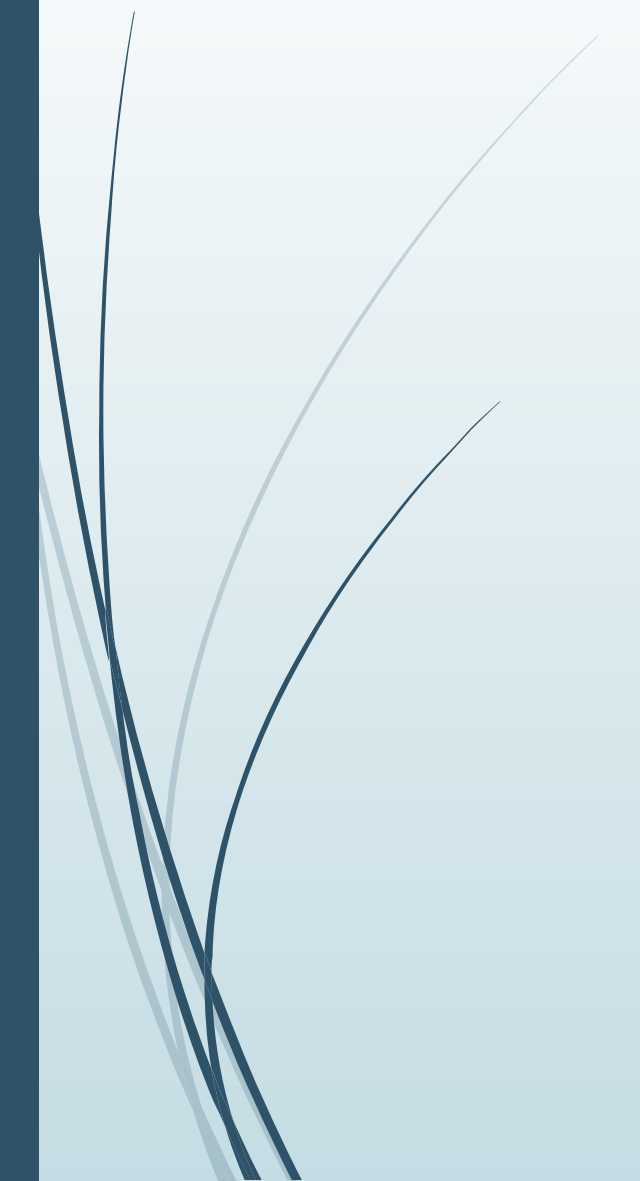
Collection widgets

- Collection widgets specialize in displaying multiple elements of the same type, such as a collection of pictures from a gallery app, a collection of articles from a news app or a collection of emails/messages from a communication app.
- Collection widgets can scroll vertically.
- Collection widgets typically focus on the following use cases:
 1. Browsing the collection
 2. Opening an element of the collection to its detail view in the associated app for consumption
 3. Interacting with elements such as marking them done (with new support for compound buttons in Android 12 (API level 31))



Control widgets

- The main purpose of a control widget is to display often-used functions, so that the user can trigger right from the home screen without having to open the app first.
- You can think of them as remote controls for an app.
- An example of a control widget is a home control widget that lets users turn lights on or off in different rooms of a house.



Light list

Bedroom

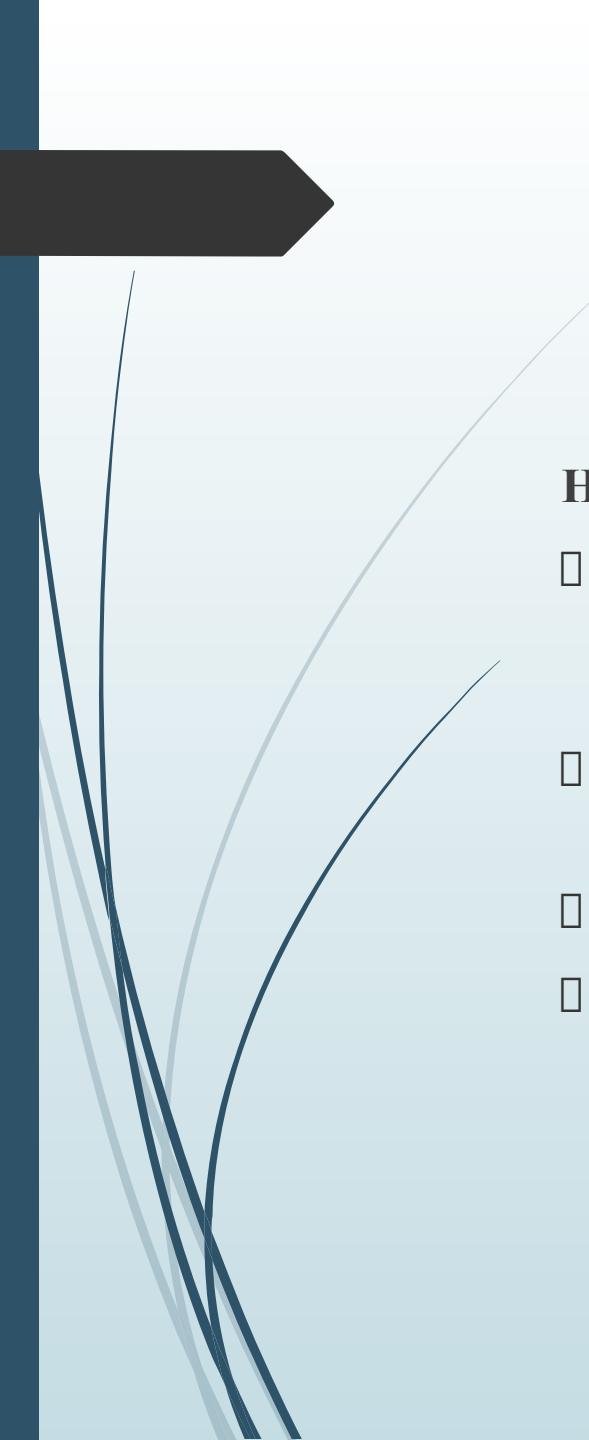


Kitchen



Living room





Hybrid widgets

- While some widgets tend to gravitate towards one of the types in the preceding sections (information, collection, or control), many widgets in reality are hybrids that combine elements of different types.
- A music player widget is primarily a control widget, but also keeps the user informed about what track is currently playing.
- It essentially combines a control widget with elements of an information widget type.
- When planning your widget, design around one of the base types and add elements of other types if needed.



Artist - Example music





Design guidelines

Widget content

- ❑ Widgets are a great mechanism to attract a user to your app by "advertising" new and interesting content that is available for consumption in your app.
- ❑ Just like the teasers on the front page of a newspaper, widgets should consolidate and concentrate an app's information and then provide a connection to richer detail within the app; or in other words: the widget is the information "snack" while the app is the "meal."
- ❑ As a bottom line, always make sure that your app shows more detail about an information item than what the widget already displays.



Widget navigation

- ❑ Besides the pure information content, consider rounding out your widget's offering by providing navigation links to frequently used areas of your app.
- ❑ This lets users complete tasks quicker and extends the functional reach of the app to the home screen.

Widget resizing

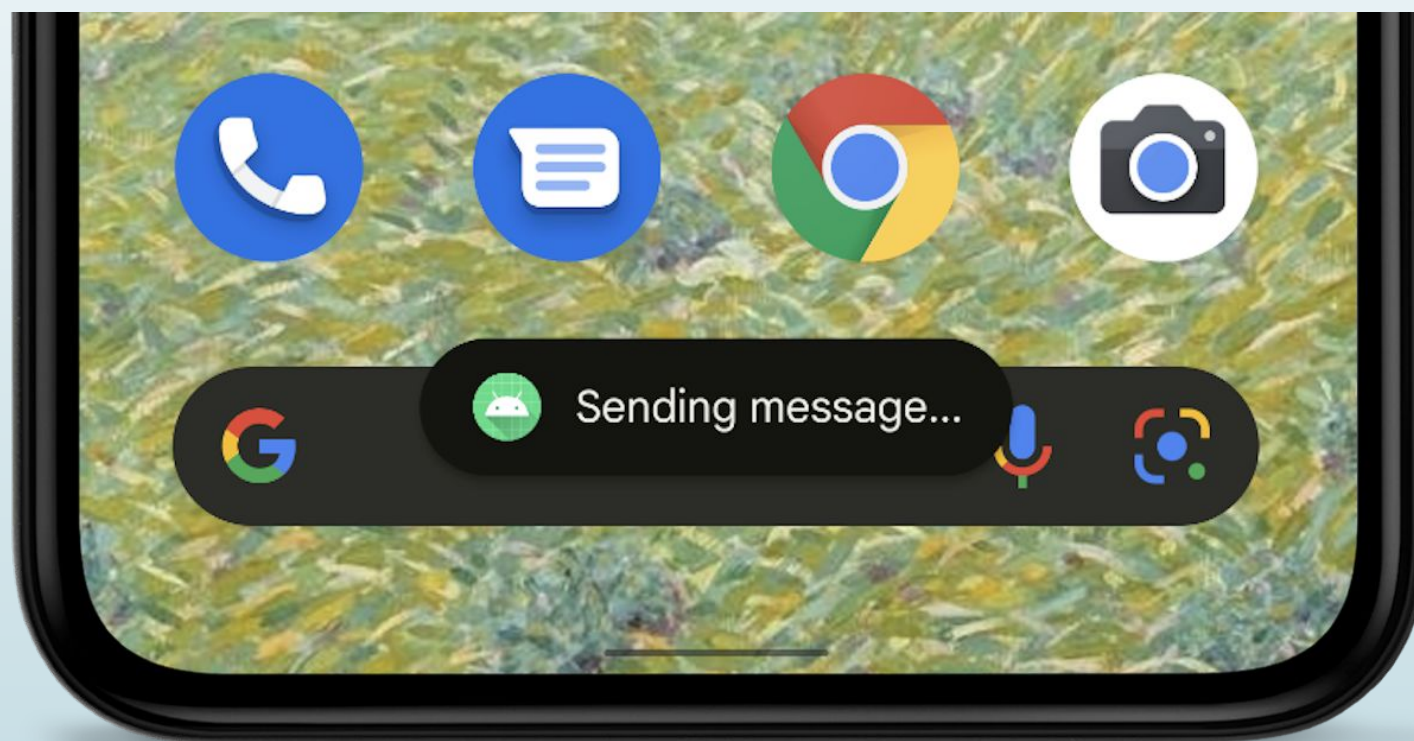
- ❑ Long-pressing on a resizable widget, and subsequently releasing puts that widget into resize mode.
- ❑ Users can use the drag handles or the widget corners to set the desired size.

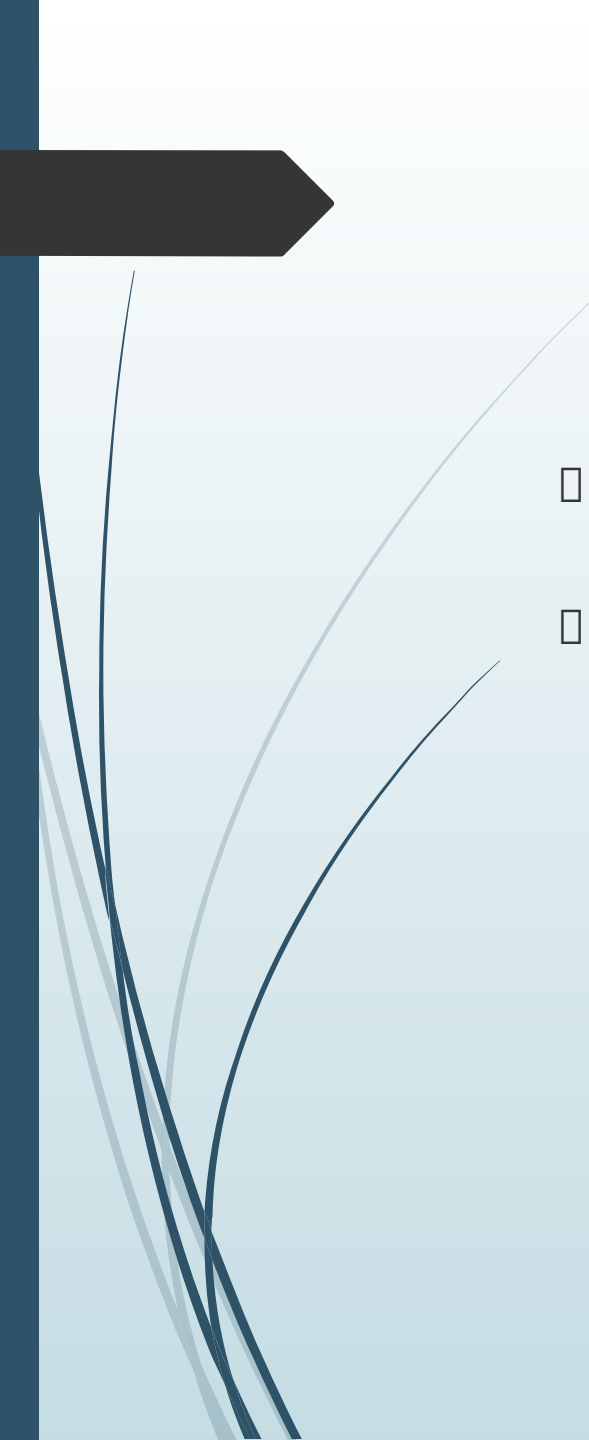


Toasts overview



- A toast provides simple **feedback** about an operation in a small popup.
- It only fills the amount of space required for the message and the current activity remains visible and interactive.
- Toasts automatically disappear after a timeout.
- For example, clicking **Send** on an email triggers a "Sending message..." toast, as shown in the following screen capture:



- 
- ❑ If your app targets Android 12 (API level 31) or higher, its toast is limited to two lines of text and shows the application icon next to the text.
 - ❑ Be aware that the line length of this text varies by screen size, so it's good to make the text as short as possible.



Instantiate a Toast object

- ❑ Use the **makeText()** method, which takes the following parameters:
- ❑ The application **Context**.
- ❑ The **text** that should appear to the user.
- ❑ The **duration** that the toast should remain on the screen.
- ❑ The **makeText()** method returns a properly initialized Toast object.



Show the toast

□ To display the toast, call the **show()** method, as demonstrated in the following example:

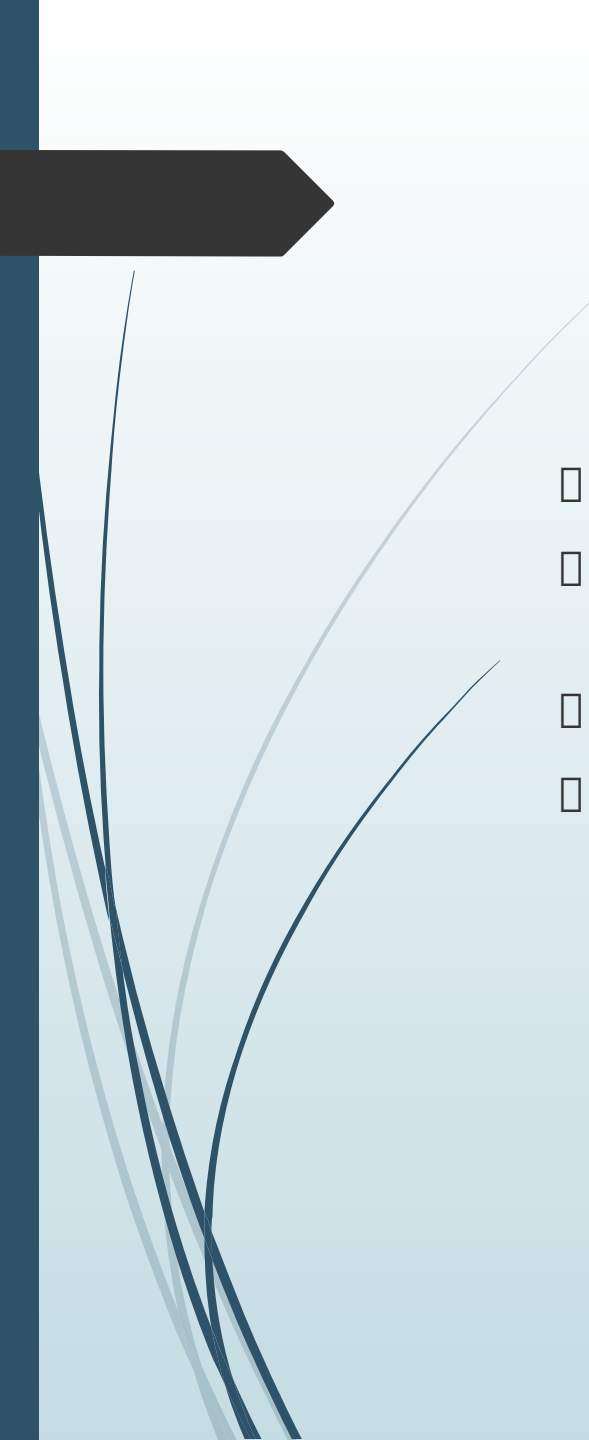
```
Context context = getApplicationContext();
```

```
CharSequence text = "Hello toast!";
```

```
int duration = Toast.LENGTH_SHORT;
```

```
Toast toast = Toast.makeText(context, text, duration);
```

```
toast.show();
```

- 
- Context is needed by **the Toast class to be able to find and use the string's resource ID.**
 - Context **provides the connection to the Android system and the resources of the project.**
 - It is the interface to global information about the application environment.
 - The Context also provides access to Android Services, e.g. the Location Service. Activities and Services extend the Context class.



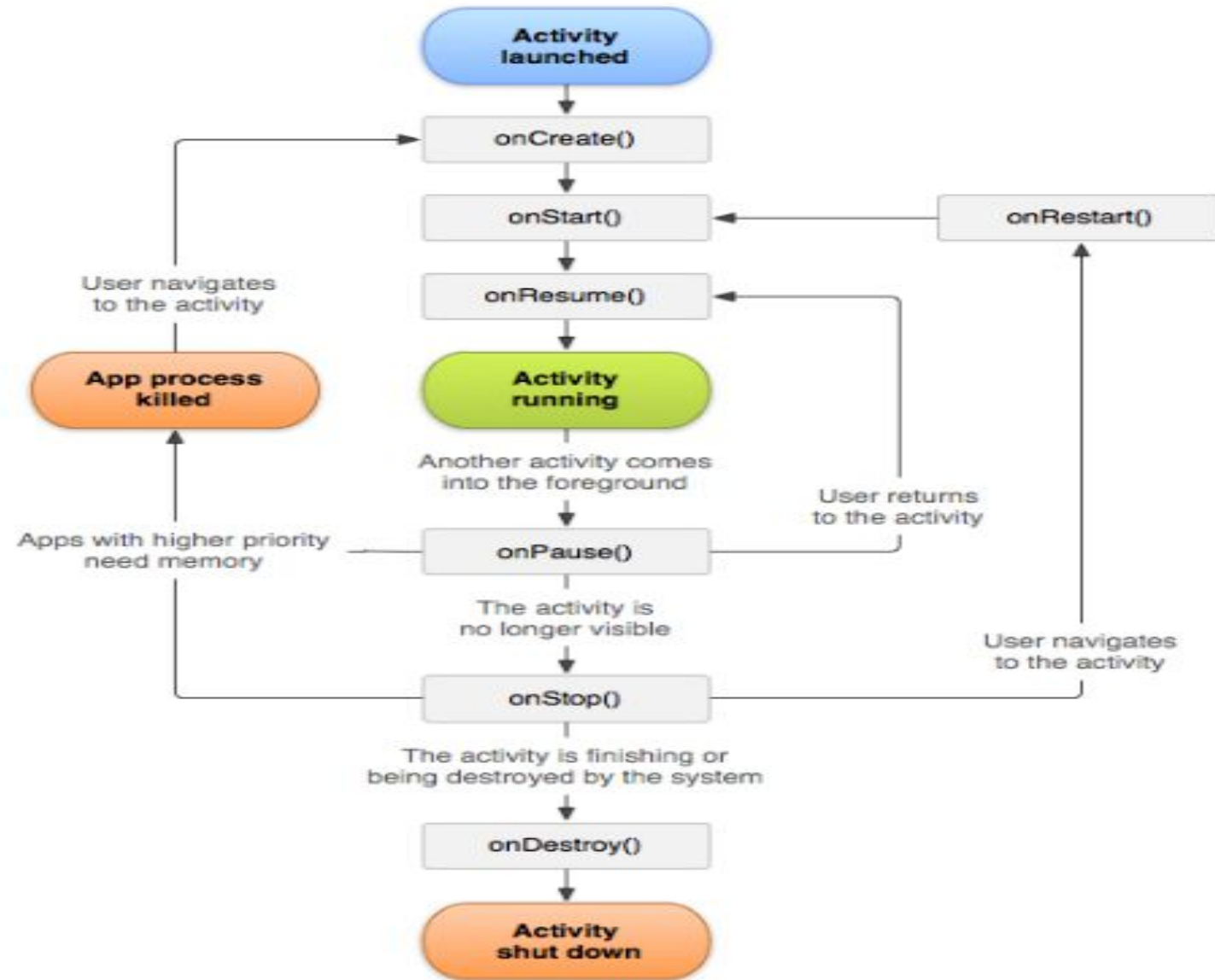
Android Activity Lifecycle



- ❑ **Android Activity Lifecycle** is controlled by 7 methods of `android.app.Activity` class.
- ❑ The android Activity is the subclass of `ContextThemeWrapper` class.
- ❑ An activity is the single screen in android. It is like window or frame of Java.
- ❑ By the help of activity, you can place all your UI components or widgets in a single screen.
- ❑ The 7 lifecycle method of Activity describes how activity will behave at different states.

Android Activity Lifecycle methods

Method	Description
onCreate	called when activity is first created.
onStart	called when activity is becoming visible to the user.
onResume	called when activity will start interacting with the user.
onPause	called when activity is not visible to the user.
onStop	called when activity is no longer visible to the user.
onRestart	called after your activity is stopped, prior to start.
onDestroy	called before the activity is destroyed.



.xml

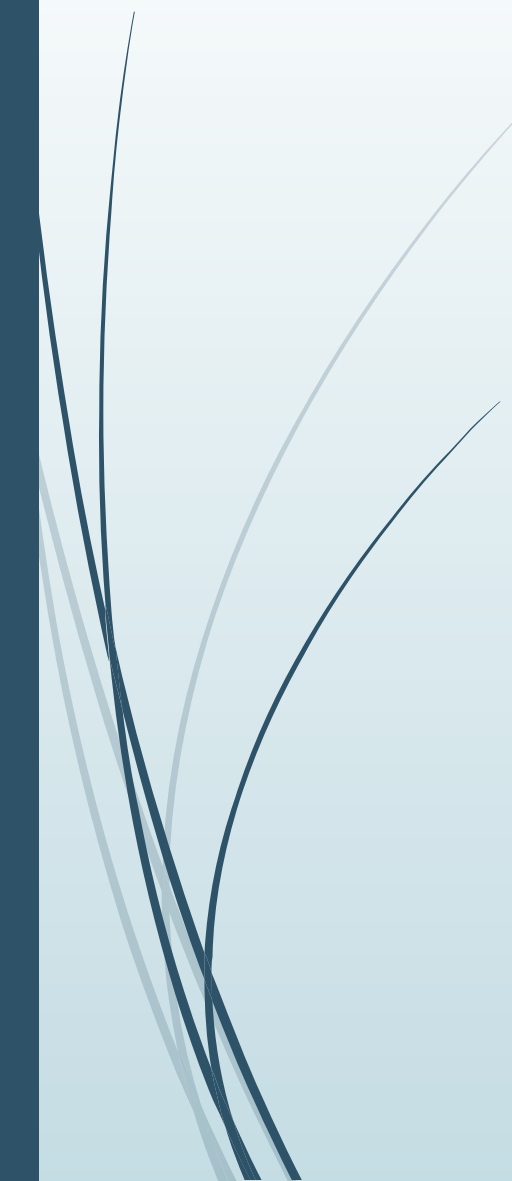
```
<?xml version="1.0" encoding="utf-8"?>
<android.support.constraint.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context="example.javatpoint.com.activitylifecycle.MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</android.support.constraint.ConstraintLayout>
```



.java



```
package example.javatpoint.com.activitylifecycle;
```

```
import android.app.Activity;
```

```
import android.os.Bundle;
```

```
import android.util.Log;
```

```
public class MainActivity extends Activity {
```

```
    @Override
```

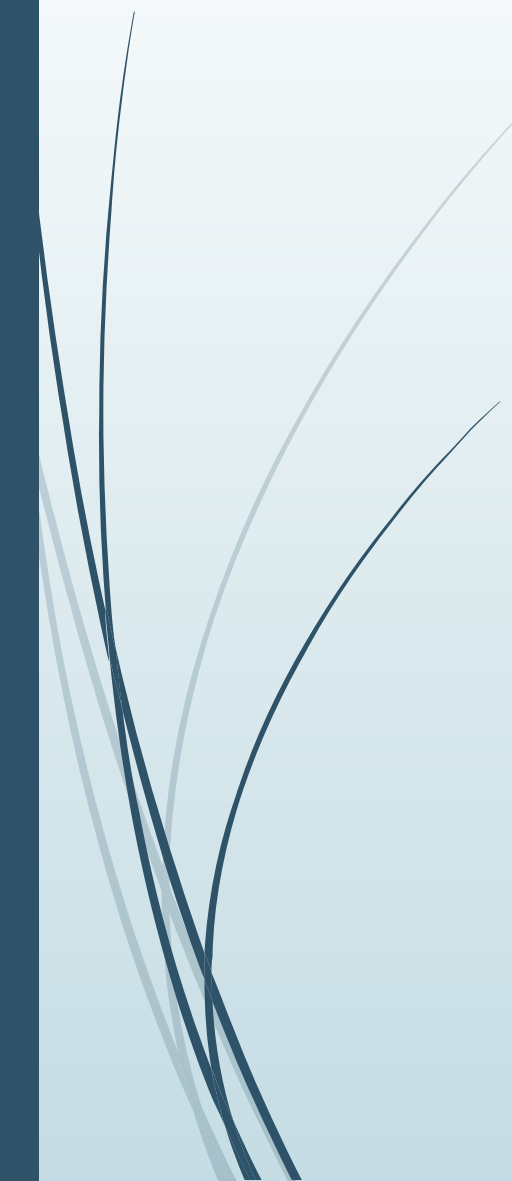
```
    protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_main);
```

```
        Log.d("lifecycle", "onCreate invoked");
```

```
    }
```



```
@Override
```

```
    protected void onStart() {
```

```
        super.onStart();
```

```
        Log.d("lifecycle","onStart invoked");
```

```
    }
```

```
@Override
```

```
protected void onResume() {
```

```
    super.onResume();
```

```
    Log.d("lifecycle","onResume invoked");
```

```
}
```

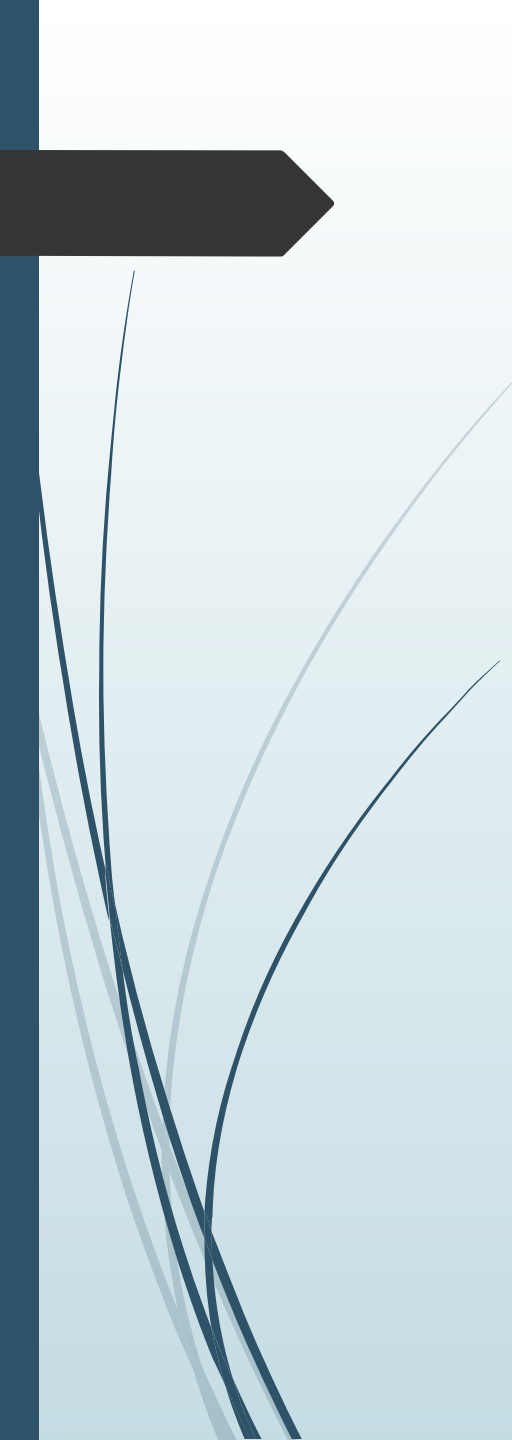
```
@Override
```

```
protected void onPause() {
```

```
    super.onPause();
```

```
    Log.d("lifecycle","onPause invoked");
```

```
}
```



```
@Override
```

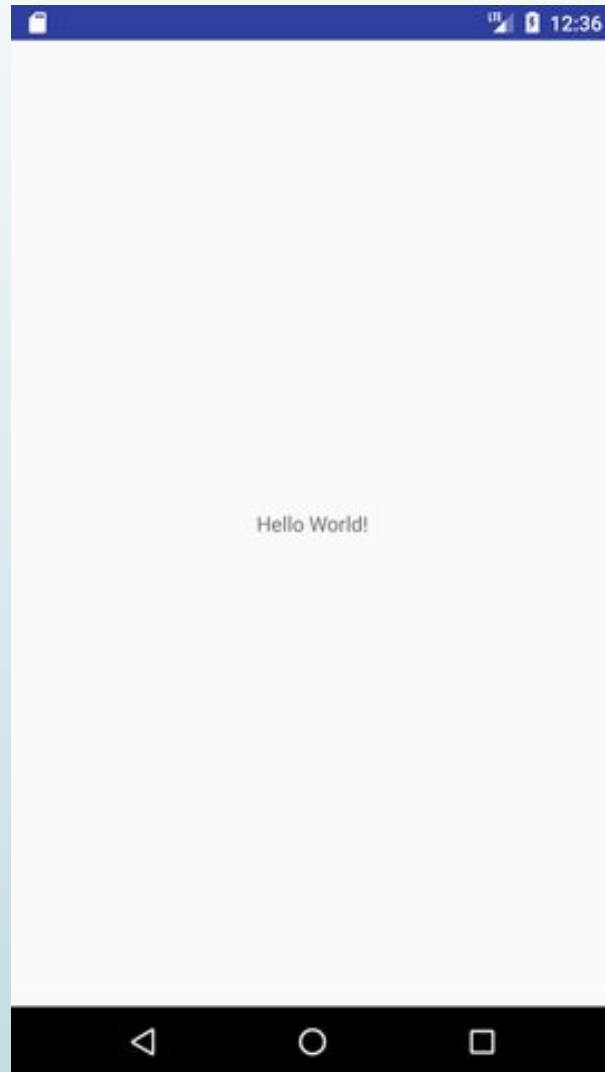
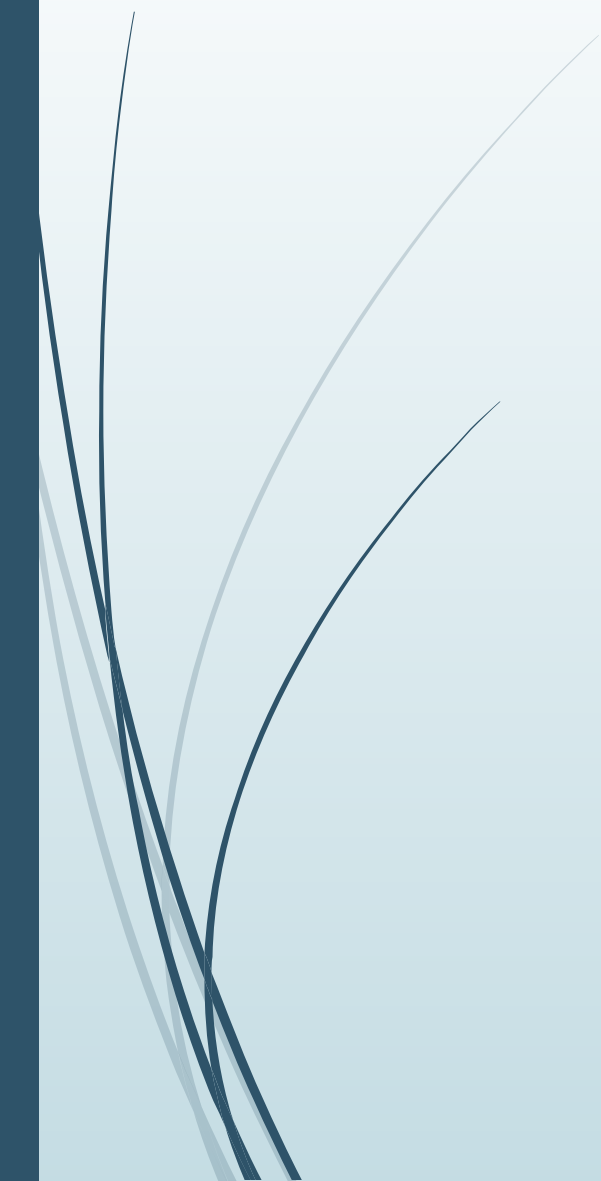
```
protected void onStop() {  
    super.onStop();  
    Log.d("lifecycle", "onStop invoked");  
}
```

```
@Override
```

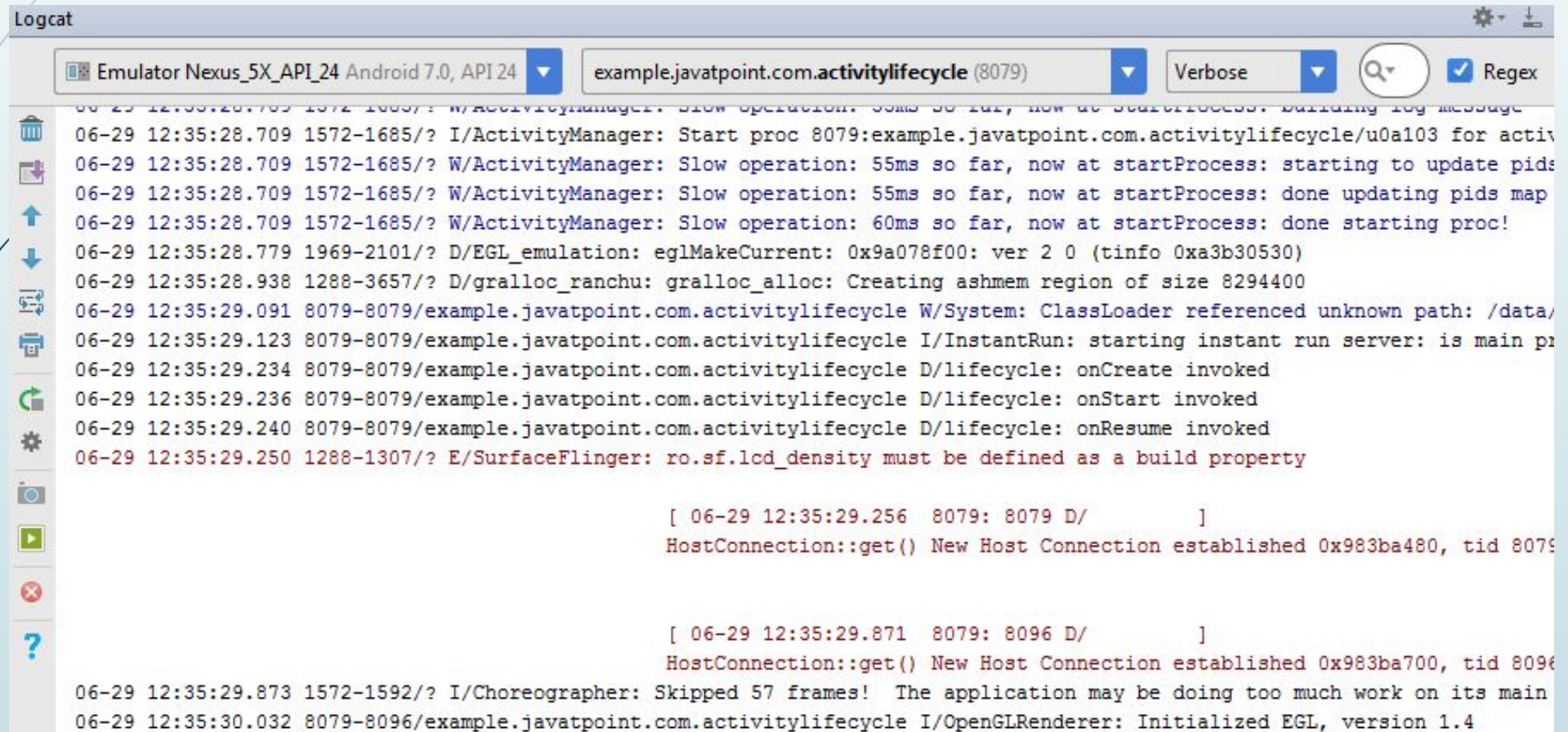
```
protected void onRestart() {  
    super.onRestart();  
    Log.d("lifecycle", "onRestart invoked");  
}
```

```
@Override
```

```
protected void onDestroy() {  
    super.onDestroy();  
    Log.d("lifecycle", "onDestroy invoked");  
}  
}
```



logcat: onCreate, onStart and onResume methods are invoked.



```
Logcat
Emulator Nexus_5X_API_24 Android 7.0, API 24 example.javatpoint.com.activitylifecycle (8079) Verbose [Q] [x] Regex

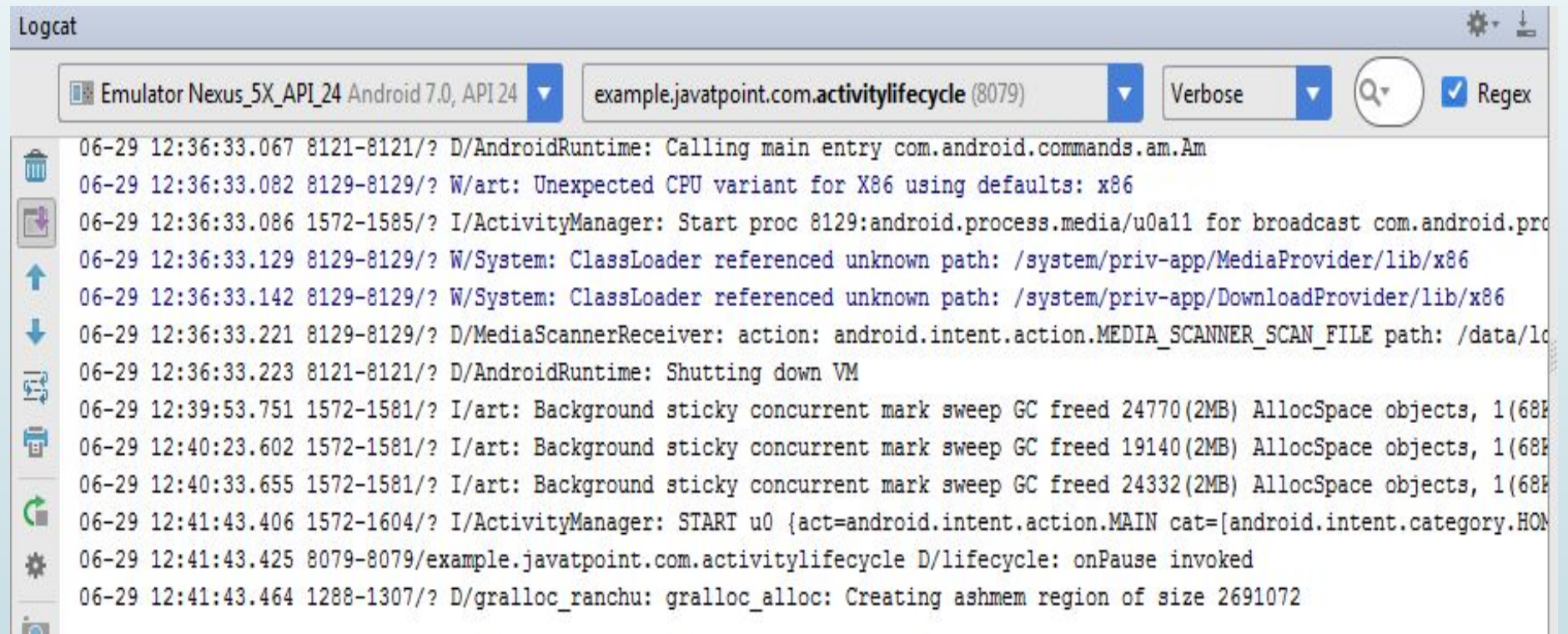
06-29 12:35:28.709 1572-1685/? W/ActivityManager: Slow operation: 55ms so far, now at startProcess: building log message
06-29 12:35:28.709 1572-1685/? I/ActivityManager: Start proc 8079:example.javatpoint.com.activitylifecycle/u0a103 for activ
06-29 12:35:28.709 1572-1685/? W/ActivityManager: Slow operation: 55ms so far, now at startProcess: starting to update pids
06-29 12:35:28.709 1572-1685/? W/ActivityManager: Slow operation: 55ms so far, now at startProcess: done updating pids map
06-29 12:35:28.709 1572-1685/? W/ActivityManager: Slow operation: 60ms so far, now at startProcess: done starting proc!
06-29 12:35:28.779 1969-2101/? D/EGL_emulation: eglMakeCurrent: 0x9a078f00: ver 2 0 (tinfo 0xa3b30530)
06-29 12:35:28.938 1288-3657/? D/gralloc_ranchu: gralloc_alloc: Creating ashmem region of size 8294400
06-29 12:35:29.091 8079-8079/example.javatpoint.com.activitylifecycle W/System: ClassLoader referenced unknown path: /data/
06-29 12:35:29.123 8079-8079/example.javatpoint.com.activitylifecycle I/InstantRun: starting instant run server: is main pr
06-29 12:35:29.234 8079-8079/example.javatpoint.com.activitylifecycle D/lifecycle: onCreate invoked
06-29 12:35:29.236 8079-8079/example.javatpoint.com.activitylifecycle D/lifecycle: onStart invoked
06-29 12:35:29.240 8079-8079/example.javatpoint.com.activitylifecycle D/lifecycle: onResume invoked
06-29 12:35:29.250 1288-1307/? E/SurfaceFlinger: ro.sf.lcd_density must be defined as a build property

[ 06-29 12:35:29.256 8079: 8079 D/
HostConnection::get() New Host Connection established 0x983ba480, tid 8079

[ 06-29 12:35:29.871 8079: 8096 D/
HostConnection::get() New Host Connection established 0x983ba700, tid 8096

06-29 12:35:29.873 1572-1592/? I/Choreographer: Skipped 57 frames! The application may be doing too much work on its main
06-29 12:35:30.032 8079-8096/example.javatpoint.com.activitylifecycle I/OpenGLRenderer: Initialized EGL, version 1.4
```

HOME Button. You will see onPause method is invoked.

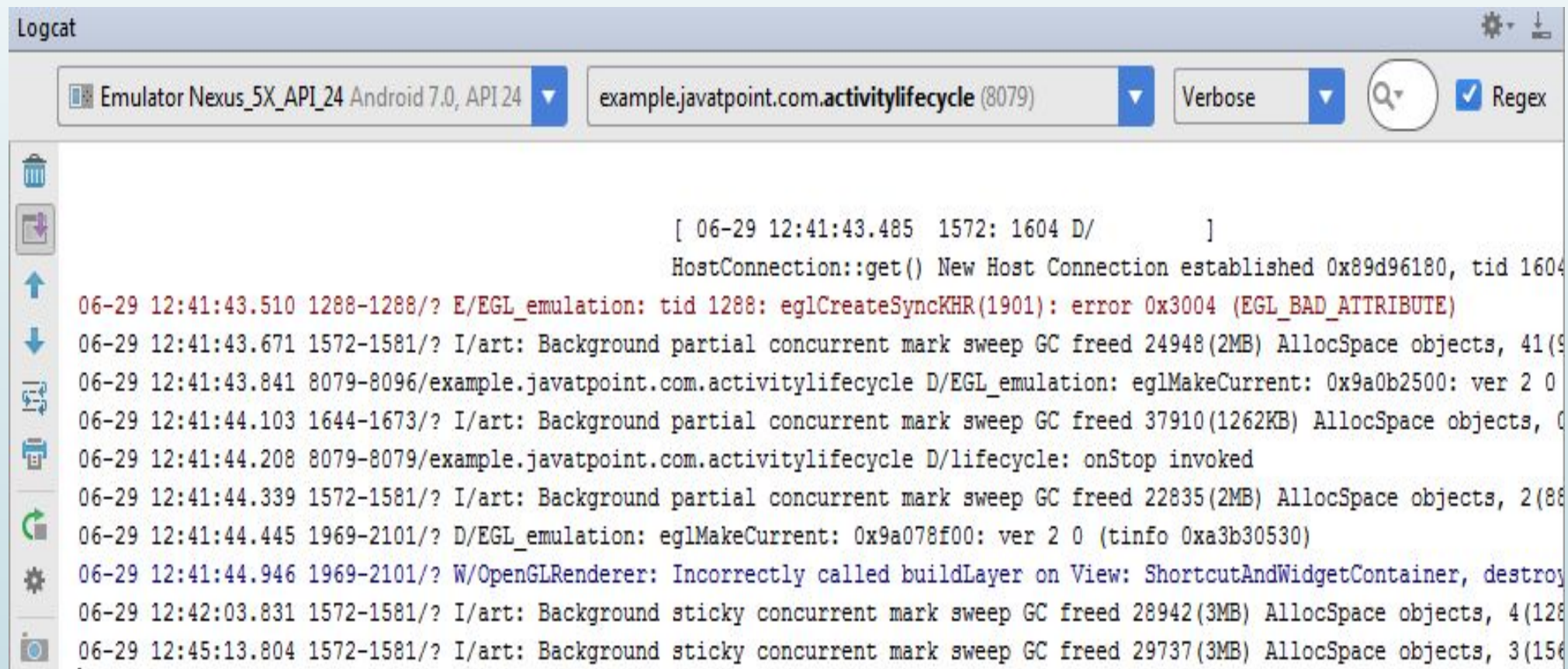


Logcat

Emulator Nexus_5X_API_24 Android 7.0, API 24 example.javatpoint.com.activitylifecycle (8079) Verbose ☒ Regex

```
06-29 12:36:33.067 8121-8121/? D/AndroidRuntime: Calling main entry com.android.commands.am.Am
06-29 12:36:33.082 8129-8129/? W/art: Unexpected CPU variant for X86 using defaults: x86
06-29 12:36:33.086 1572-1585/? I/ActivityManager: Start proc 8129:android.process.media/u0a11 for broadcast com.android.pro
06-29 12:36:33.129 8129-8129/? W/System: ClassLoader referenced unknown path: /system/priv-app/MediaProvider/lib/x86
06-29 12:36:33.142 8129-8129/? W/System: ClassLoader referenced unknown path: /system/priv-app/DownloadProvider/lib/x86
06-29 12:36:33.221 8129-8129/? D/MediaScannerReceiver: action: android.intent.action.MEDIA_SCANNER_SCAN_FILE path: /data/lo
06-29 12:36:33.223 8121-8121/? D/AndroidRuntime: Shutting down VM
06-29 12:39:53.751 1572-1581/? I/art: Background sticky concurrent mark sweep GC freed 24770(2MB) AllocSpace objects, 1(68K
06-29 12:40:23.602 1572-1581/? I/art: Background sticky concurrent mark sweep GC freed 19140(2MB) AllocSpace objects, 1(68K
06-29 12:40:33.655 1572-1581/? I/art: Background sticky concurrent mark sweep GC freed 24332(2MB) AllocSpace objects, 1(68K
06-29 12:41:43.406 1572-1604/? I/ActivityManager: START u0 {act=android.intent.action.MAIN cat=[android.intent.category.HOM
06-29 12:41:43.425 8079-8079/example.javatpoint.com.activitylifecycle D/lifecycle: onPause invoked
06-29 12:41:43.464 1288-1307/? D/gralloc_ranchu: gralloc_alloc: Creating ashmem region of size 2691072
```


After a while, you will see onStop method is invoked.



```
Logcat
Emulator Nexus_5X_API_24 Android 7.0, API 24 example.javatpoint.com.activitylifecycle (8079) Verbose [Q] [x] Regex

[ 06-29 12:41:43.485 1572: 1604 D/      ]
HostConnection::get() New Host Connection established 0x89d96180, tid 1604
06-29 12:41:43.510 1288-1288/? E/EGL_emulation: tid 1288: eglCreateSyncKHR(1901): error 0x3004 (EGL_BAD_ATTRIBUTE)
06-29 12:41:43.671 1572-1581/? I/art: Background partial concurrent mark sweep GC freed 24948(2MB) AllocSpace objects, 41(9
06-29 12:41:43.841 8079-8096/example.javatpoint.com.activitylifecycle D/EGL_emulation: eglMakeCurrent: 0x9a0b2500: ver 2 0
06-29 12:41:44.103 1644-1673/? I/art: Background partial concurrent mark sweep GC freed 37910(1262KB) AllocSpace objects, 0
06-29 12:41:44.208 8079-8079/example.javatpoint.com.activitylifecycle D/lifecycle: onStop invoked
06-29 12:41:44.339 1572-1581/? I/art: Background partial concurrent mark sweep GC freed 22835(2MB) AllocSpace objects, 2(88
06-29 12:41:44.445 1969-2101/? D/EGL_emulation: eglMakeCurrent: 0x9a078f00: ver 2 0 (tinfo 0xa3b30530)
06-29 12:41:44.946 1969-2101/? W/OpenGLRenderer: Incorrectly called buildLayer on View: ShortcutAndWidgetContainer, destroy
06-29 12:42:03.831 1572-1581/? I/art: Background sticky concurrent mark sweep GC freed 28942(3MB) AllocSpace objects, 4(128
06-29 12:45:13.804 1572-1581/? I/art: Background sticky concurrent mark sweep GC freed 29737(3MB) AllocSpace objects, 3(156
```



Android Intent



- ❑ **Android Intent** is the *message* that is passed between components such as **activities, content providers, broadcast receivers, services** etc.
- ❑ It is generally used with **startActivity()** method to invoke activity, broadcast receivers etc.
- ❑ It can be described as the intention to do action.
- ❑ Used for communication between system and application, multiple activities.



Android intents are mainly used to:

- ❑ Start the service
- ❑ Launch an activity
- ❑ Display a web page
- ❑ Display a list of contacts
- ❑ Broadcast a message
- ❑ Dial a phone call etc.

Types of Android Intents

- There are two types of intents in android: implicit and explicit.

1) Implicit Intent

- **Implicit Intent** doesn't specify the component. In such case, intent provides information of available components provided by the system that is to be invoked.
- Ex. Location in app
- For example, you may write the following code to view the webpage.

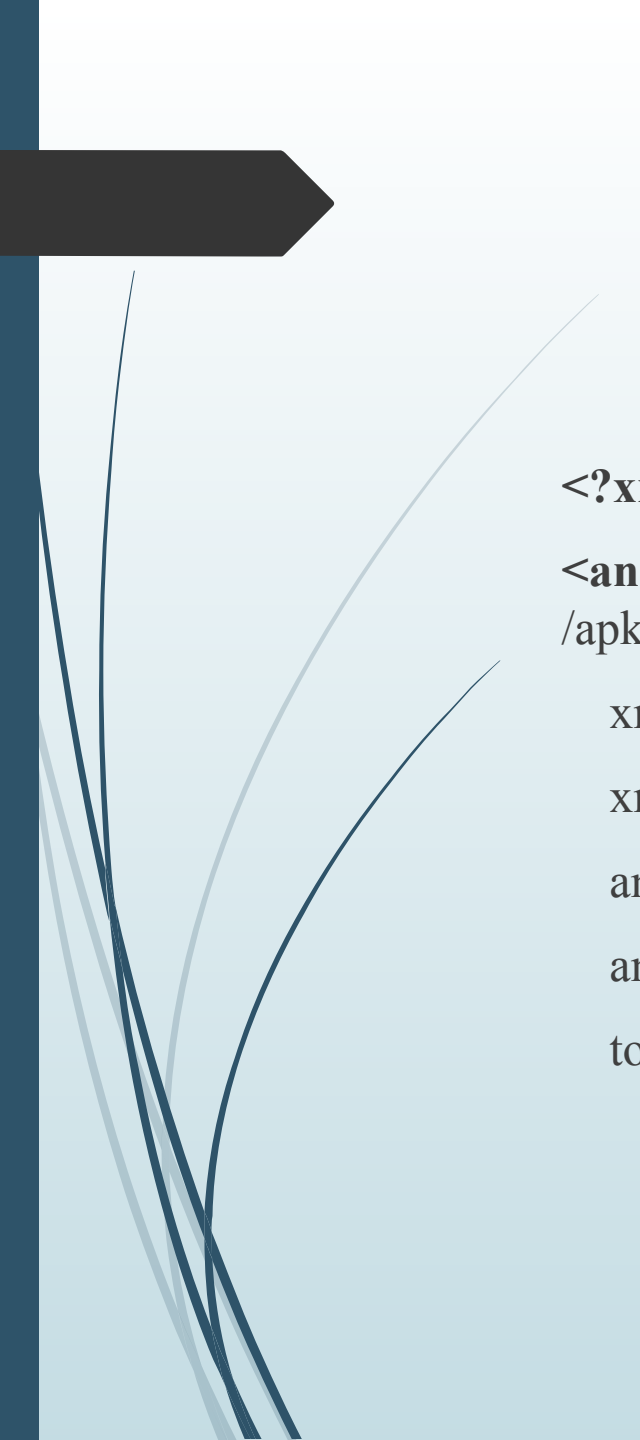
```
Intent intent=new Intent(Intent.ACTION_VIEW);  
intent.setData(Uri.parse("http://www.google.com"));  
startActivity(intent);
```



❑ 2) Explicit Intent

- ❑ **Explicit Intent** specifies the component. In such case, intent provides the external class to be invoked.
- ❑ If we know the name of second activity, then we can move from one activity to another activity.
- ❑ Ex. Login page

```
Intent i = new Intent(getApplicationContext(), ActivityTwo.class);  
startActivity(i);
```



```
<?xml version="1.0" encoding="utf-8"?>
```

```
<android.support.constraint.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    xmlns:app="http://schemas.android.com/apk/res-auto"
```

```
    xmlns:tools="http://schemas.android.com/tools"
```

```
    android:layout_width="match_parent"
```

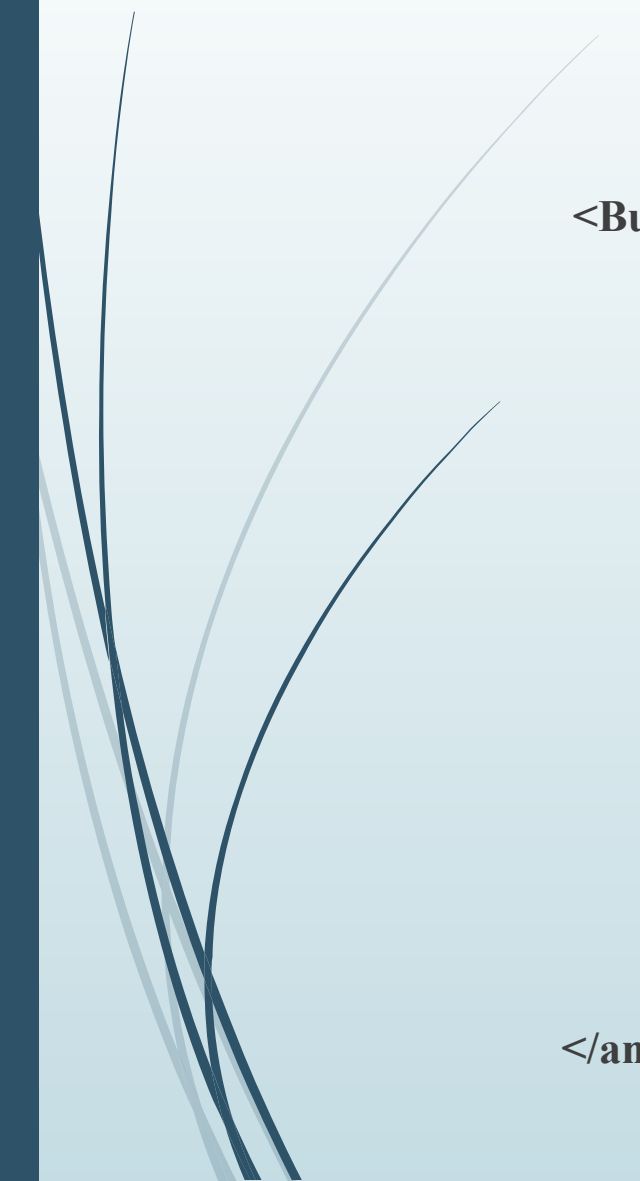
```
    android:layout_height="match_parent"
```

```
    tools:context="example.javatpoint.com.implicitintent.MainActivity">
```




<EditText

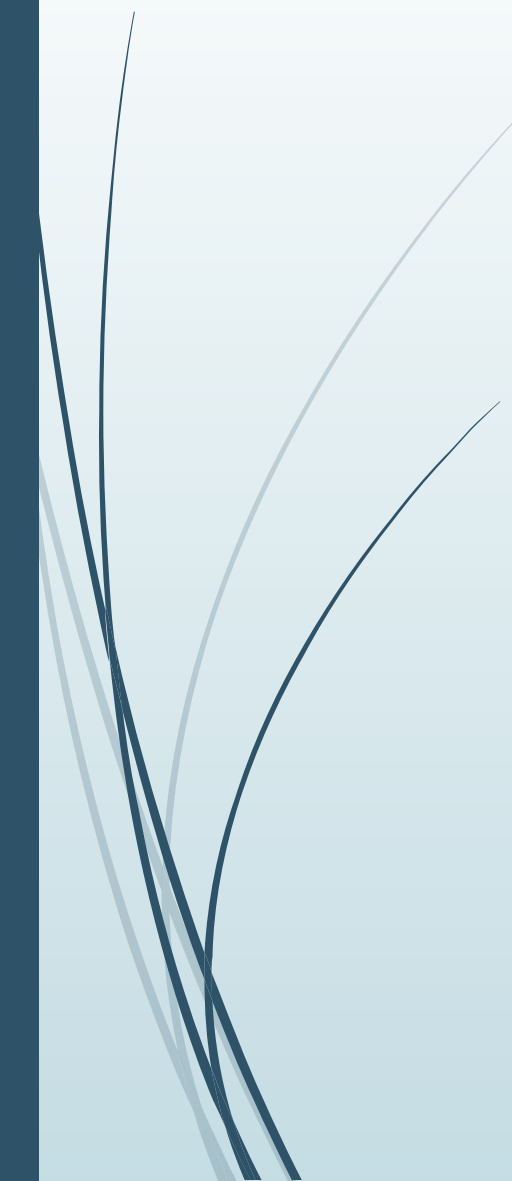
```
    android:id="@+id/editText"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:layout_marginEnd="8dp"  
    android:layout_marginStart="8dp"  
    android:layout_marginTop="60dp"  
    android:ems="10"  
    app:layout_constraintEnd_toEndOf="parent"  
    app:layout_constraintHorizontal_bias="0.575"  
    app:layout_constraintStart_toStartOf="parent"  
    app:layout_constraintTop_toTopOf="parent" />
```



<Button

```
    android:id="@+id/button"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:layout_marginRight="8dp"  
    android:layout_marginLeft="156dp"  
    android:layout_marginTop="172dp"  
    android:text="Visit"  
    app:layout_constraintEnd_toEndOf="parent"  
    app:layout_constraintHorizontal_bias="0.0"  
    app:layout_constraintStart_toStartOf="parent"  
    app:layout_constraintTop_toBottomOf="@+id/editText" />
```

</android.support.constraint.ConstraintLayout>



```
package example.javatpoint.com.implicitintent;
```

```
import android.content.Intent;
```

```
import android.net.Uri;
```

```
import android.support.v7.app.AppCompatActivity;
```

```
import android.os.Bundle;
```

```
import android.view.View;
```

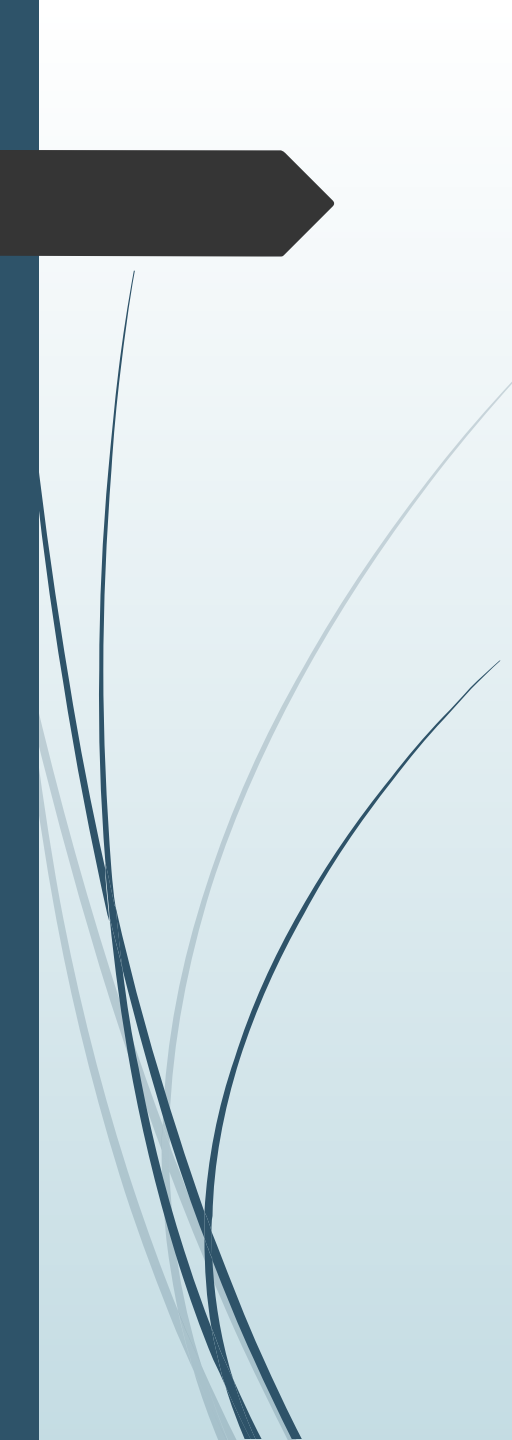
```
import android.widget.Button;
```

```
import android.widget.EditText;
```

```
public class MainActivity extends AppCompatActivity {
```

```
    Button button;
```

```
    EditText editText;
```



```
@Override
```

```
protected void onCreate(Bundle savedInstanceState) {
```

```
    super.onCreate(savedInstanceState);
```

```
    setContentView(R.layout.activity_main);
```

```
    button = findViewById(R.id.button);
```

```
    editText = findViewById(R.id.editText);
```

```
    button.setOnClickListener(new View.OnClickListener() {
```

```
        @Override
```

```
        public void onClick(View view) {
```

```
            String url=editText.getText().toString();
```

```
            Intent intent=new Intent(Intent.ACTION_VIEW, Uri.parse(url));
```

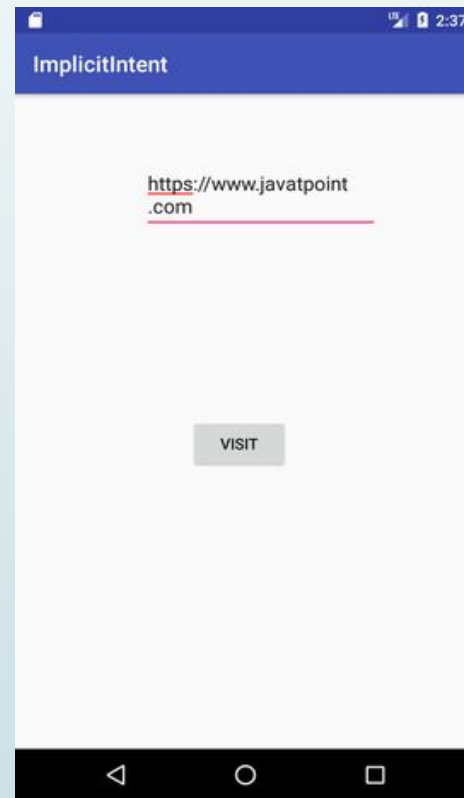
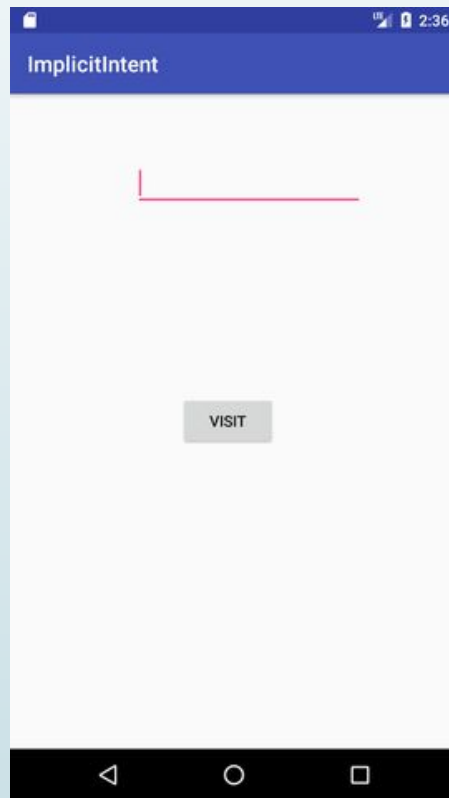
```
            startActivity(intent);
```

```
        }
```

```
    });
```

```
}
```

```
}
```



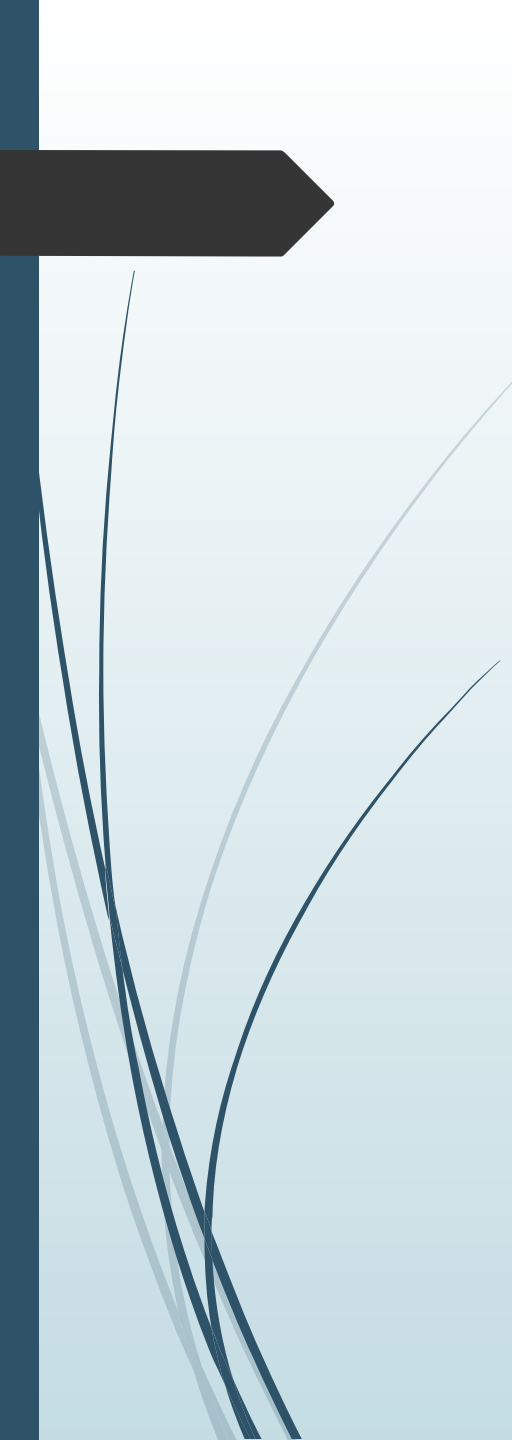


Android Explicit Intent

- ❑ **Android Explicit intent** specifies the component to be invoked from activity. In other words, we can call another activity in android by explicit intent.
- ❑ We can also pass the information from one activity to another using explicit intent.
- ❑ Here, we are going to see an example to call one activity from another and vice-versa.



Android calling one activity from another activity example



```
<?xml version="1.0" encoding="utf-8"?>
```

```
<android.support.constraint.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    xmlns:app="http://schemas.android.com/apk/res-auto"
```

```
    xmlns:tools="http://schemas.android.com/tools"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    tools:context="example.javatpoint.com.explicitintent.FirstActivity">
```

```
    <TextView
```

```
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
```

```
        android:layout_marginEnd="8dp"
```

```
        android:layout_marginStart="8dp"
```

```
        android:layout_marginTop="8dp"
```

```
        android:text="First Activity"
```

```
        app:layout_constraintBottom_toBottomOf="parent"
```

```
        app:layout_constraintEnd_toEndOf="parent"
```

```
        app:layout_constraintHorizontal_bias="0.454"
```

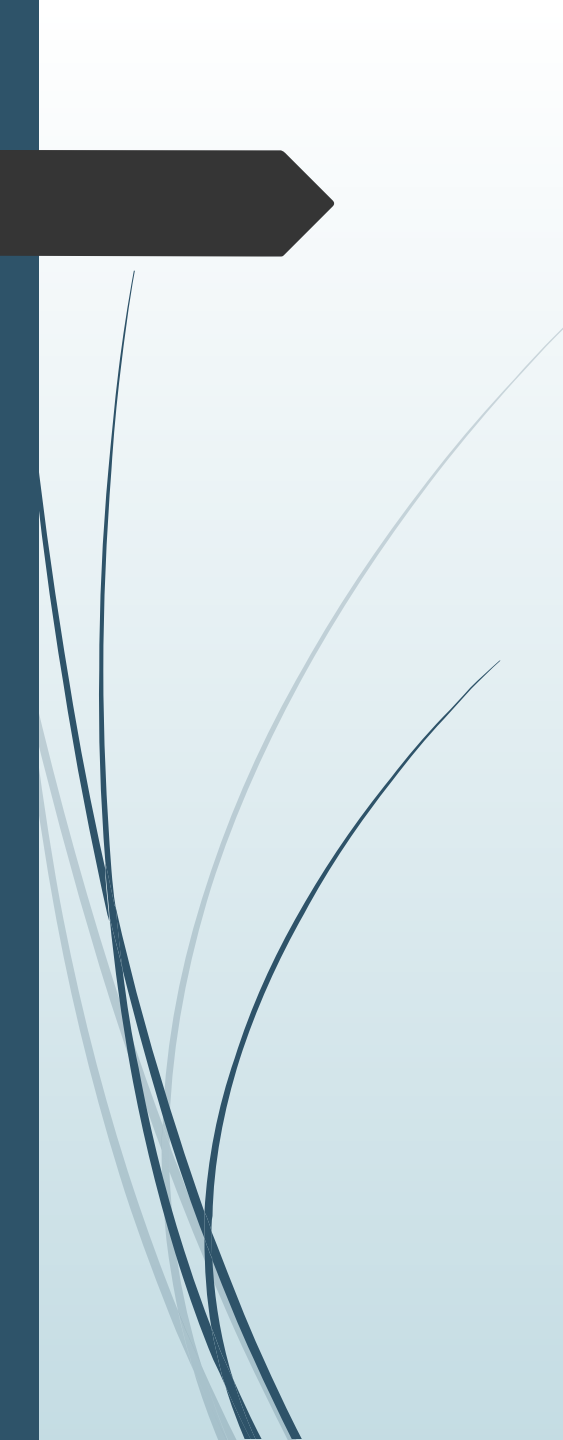
```
        app:layout_constraintLeft_toLeftOf="parent"
```

```
        app:layout_constraintRight_toRightOf="parent"
```

```
        app:layout_constraintStart_toStartOf="parent"
```

```
        app:layout_constraintTop_toTopOf="parent"
```

```
        app:layout_constraintVertical_bias="0.06" />
```

```
<Button
```

```
    android:id="@+id/button"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:layout_marginEnd="8dp"  
    android:layout_marginStart="8dp"  
    android:layout_marginTop="392dp"  
    android:onClick="callSecondActivity"  
    android:text="Call second activity"  
    app:layout_constraintEnd_toEndOf="parent"  
    app:layout_constraintStart_toStartOf="parent"  
    app:layout_constraintTop_toTopOf="parent" />
```

```
</android.support.constraint.ConstraintLayout>
```



```
package example.javatpoint.com.explicitintent;
```

```
import android.content.Intent;
```

```
import android.support.v7.app.AppCompatActivity;
```

```
import android.os.Bundle;
```

```
import android.view.View;
```

```
public class FirstActivity extends AppCompatActivity {
```

```
    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_first);
```

```
    }
```

```
    public void callSecondActivity(View view){
```

```
        Intent i = new Intent(getApplicationContext(), SecondActivity.class);
```

```
        i.putExtra("Value1", "Android By Javatpoint");
```

```
        i.putExtra("Value2", "Simple Tutorial");
```

```
        // Set the request code to any code you like, you can identify the
```

```
        // callback via this code
```

```
        startActivity(i);
```

```
    }
```



```
<?xml version="1.0" encoding="utf-8"?>
<android.support.constraint.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context="example.javatpoint.com.explicitintent.SecondActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginEnd="8dp"
        android:layout_marginStart="8dp"
        android:layout_marginTop="8dp"
        android:text="Second Activity"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.454"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.06" />
```



```
<Button
```

```
    android:id="@+id/button"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:layout_marginEnd="8dp"
```

```
    android:layout_marginStart="8dp"
```

```
    android:layout_marginTop="392dp"
```

```
    android:onClick="callFirstActivity"
```

```
    android:text="Call first activity"
```

```
    app:layout_constraintEnd_toEndOf="parent"
```

```
    app:layout_constraintStart_toStartOf="parent"
```

```
    app:layout_constraintTop_toTopOf="parent" />
```

```
</android.support.constraint.ConstraintLayout>
```



```
package example.javatpoint.com.explicitintent;

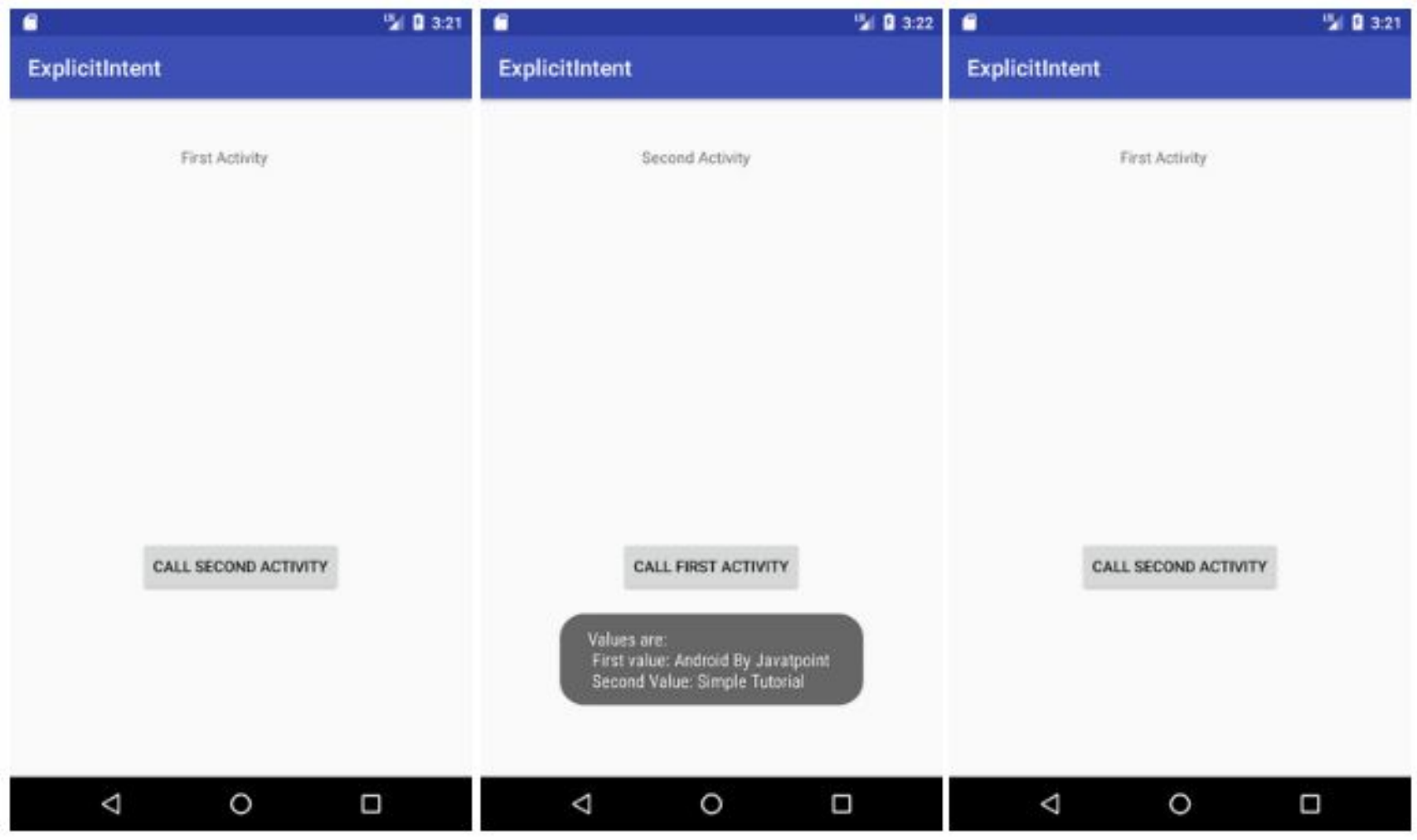
import android.content.Intent;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.Toast;

public class SecondActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);
        Bundle extras = getIntent().getExtras();
        String value1 = extras.getString("Value1");
        String value2 = extras.getString("Value2");
        Toast.makeText(getApplicationContext(), "Values are:\n First value: "+value1+
            "\n Second Value: "+value2, Toast.LENGTH_LONG).show();
    }

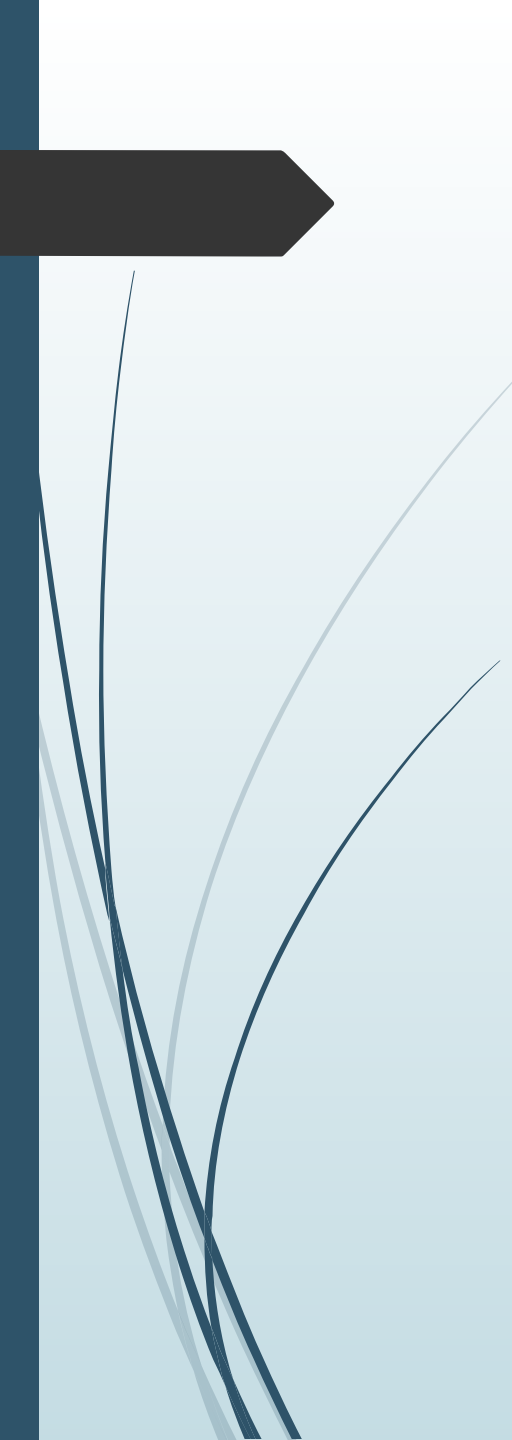
    public void callFirstActivity(View view){
        Intent i = new Intent(getApplicationContext(), FirstActivity.class);
        startActivity(i);
    }

}
```





Fundamentals of testing Android apps



Benefits of testing

- Testing is an integral part of the app development process.
- By running tests against your app consistently, you can verify your app's **correctness**, **functional behavior**, and usability before you release it publicly.
- You can *manually* test your app by navigating through it.
- You might use different devices and emulators, change the system language, and try to generate every user error or traverse every user flow.

Types of tests in Android

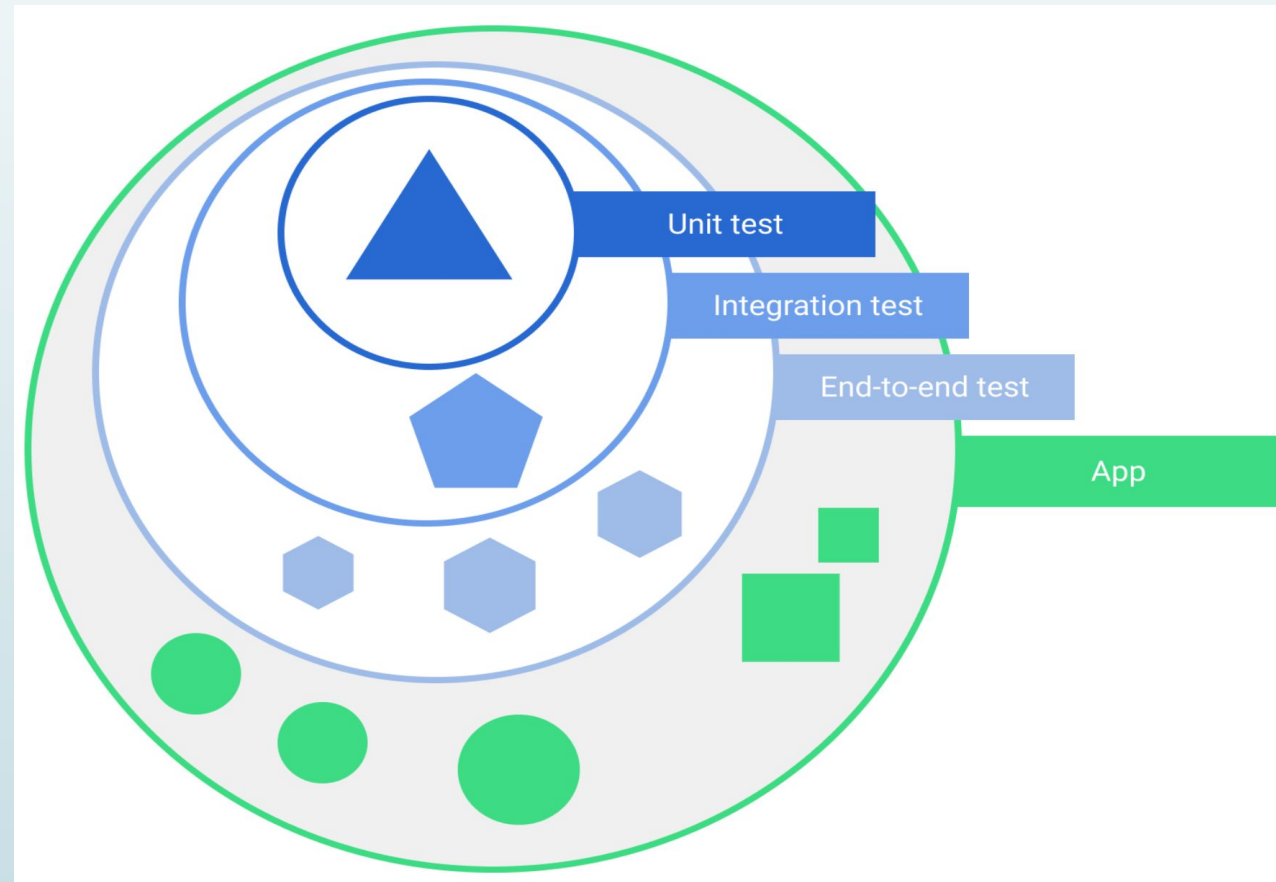
Subject

- There are different types of tests depending on the *subject*:
- **Functional testing**: does my app do what it's supposed to?
- **Performance testing**: does it do it quickly and efficiently?
- **Accessibility testing**: does it work well with accessibility services?
- **Compatibility testing**: does it work well on every device and API level?



Scope

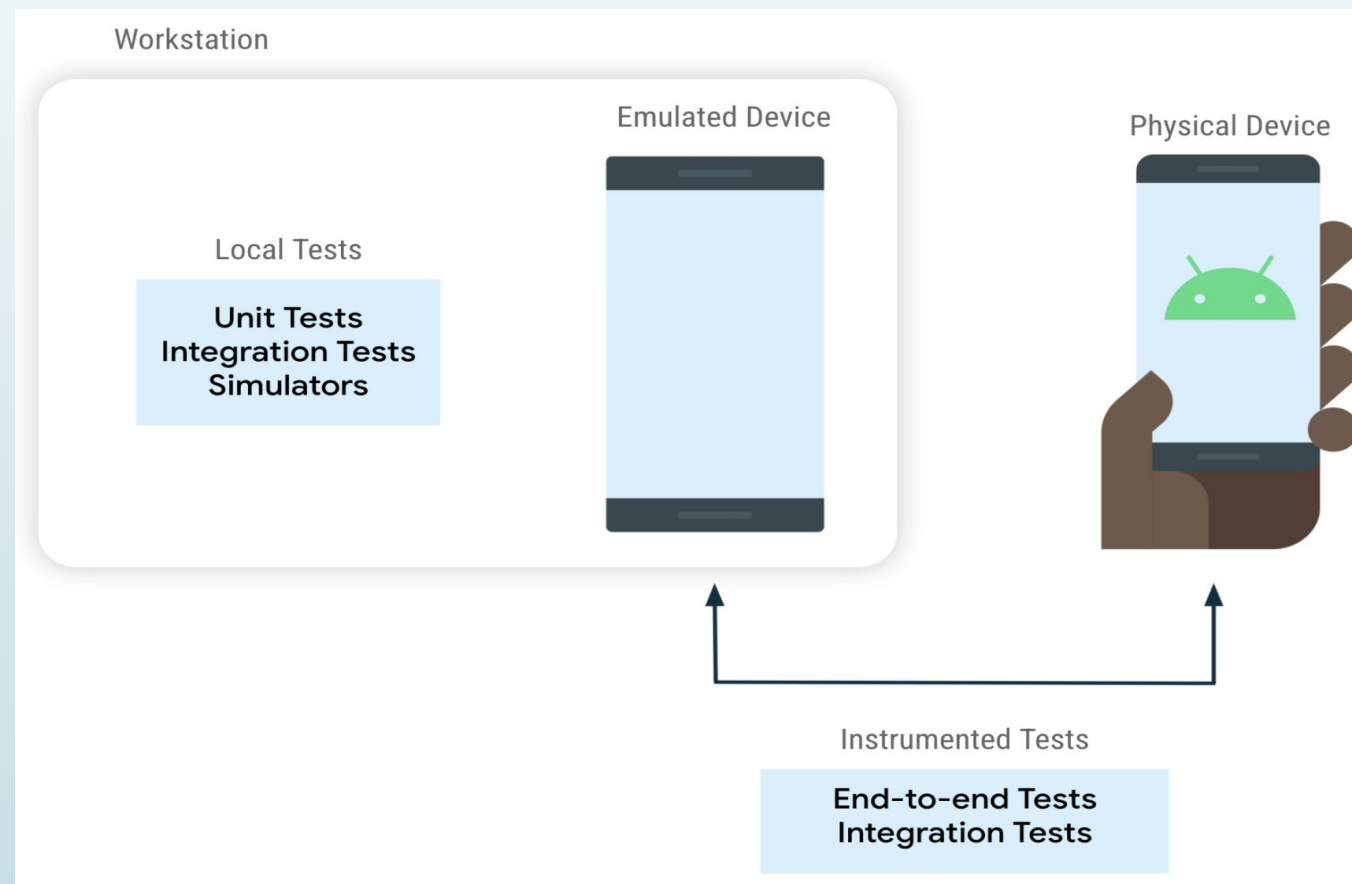
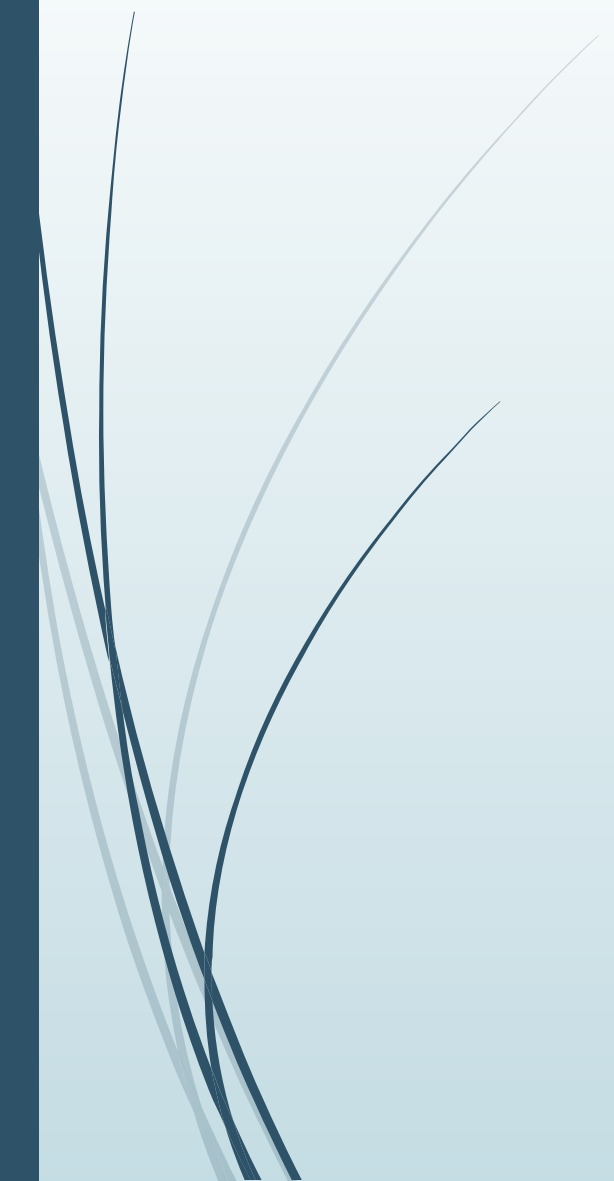
- Tests also vary depending on *size*, or *degree of isolation*:
- **Unit tests** or **small tests** only verify a very small portion of the app, such as a method or class.
- **End-to-end** tests or **big tests** verify larger parts of the app at the same time, such as a whole screen or user flow.
- **Medium tests** are in between and check the **integration** between two or more units.





Instrumented versus local tests

- ❑ **Instrumented tests** run on an Android device, either physical or emulated.
- ❑ The app is built and installed alongside a *test app* that injects commands and reads the state. Instrumented tests are usually UI tests, launching an app and then interacting with it.
- ❑ **Local tests** execute on your development machine or a server, so they're also called *host-side tests*. They're usually small and fast, isolating the subject under test from the rest of the app.





Debug your app

- ❑ Android Studio provides a debugger that allows you to do the following and more:
- ❑ Select a device to debug your app on.
- ❑ Set breakpoints in your Java, Kotlin, and C/C++ code.
- ❑ Examine variables and evaluate expressions at runtime.